



TANSZÉKVEZETŐ

SZAKDOLGOZAT FELADAT

Csutár Márk Tibor

Mérnök-informatikus hallgató részére

Orvosi adatok feldolgozása gépi tanulással

A mélytanulás lehetővé teszi, hogy orvosi adatokon olyan elváltozásokat is felismerjünk, amiket a tradicionális módszereken túlmutatnak. Azonban az orvosi adatok feldolgozása speciális körülményt igényelnek, nem lehet a képfeldolgozásban alkalmazott neurális hálókat egy az egyben átültetni. Nagyon fontos az előfeldolgozás, illetve az adatoknak a kiértékelése.

A hallgató feladata ilyen orvosi, mélytanulás alapú algoritmusok tervezése, megvalósítása és kiértékelése. A hallgató munkája során elsősorban melanóma felvételekkel dolgozik.

A hallgató feladatának a következőkre kell kiterjednie:

- Mutassa be speciálisan az orvosi adatokat! Elemezze az adatokat!
- Ismertesse a képfeldolgozásnál használt neurális háló architektúrákat! Csoportosítsa ezeket! Készítsen összehasonlítást előnyeikről és hátrányaikról!
- Tervezen és implementáljon neurális háló architektúrát orvosi adatok feldolgozására!
- Tesztelje az architektúrát és elemezze az eredményeket!

Tanszéki konzulens: Dr. Szegletes Luca, egyetemi adjunktus

Budapest, 2026. 03. 16.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető





Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Automatizálási és Alkalmazott Informatikai Tanszék

Csutár Márk Tibor

ORVOSI ADATOK FELDOLGOZÁSA GÉPI TANULÁSSA

KONZULENS

Dr. Szegletes Luca

BUDAPEST, 2026

Tartalomjegyzék

Összefoglaló.....	6
Abstract.....	7
1 Bevezetés.....	8
2 Irodalomkutatás és technológiák.....	9
2.1 Klasszikus gépi képfelismerés.....	9
2.2 Kép előfeldolgozási technikák.....	10
2.2.1 Általános előfeldolgozási lépések.....	10
2.2.2 Kernel konvolúció.....	10
2.3 Neurális hálók képfelismerésre.....	11
2.3.1 Konvolucionális Neurális Hálók.....	11
2.4 Transformerek képfelismerésre.....	14
2.4.1 Eredeti transzformer architektúra.....	14
2.4.2 Vision Transformer.....	18
2.4.3 Swin Transzformer.....	19
2.5 Hibrid architektúrák.....	21
3 Tervezés.....	23
3.1 Technológiai háttér.....	23
3.1.1 Számítási platform.....	23
3.1.2 Programozási nyelv és keretrendszerek.....	23
3.1.3 Egyéb szoftverek és technológiák.....	25
3.2 Adathalmaz és elemzése.....	25
3.2.1 Vizsgálati módszerek.....	26
3.2.2 Eredmények.....	27
3.2.3 Módosított SLCD adathalmaz.....	30
3.3 Háló tervezés.....	33
3.3.1 Választott próba hálók.....	33
3.4 Tanítás.....	34
3.4.1 Előfeldolgozási lépések.....	34
3.4.2 Súlyozott veszteségfüggvény.....	35
3.4.3 Optimalizáló.....	35

3.4.4 Hiperparaméterek.....	36
3.4.5 Értékelési metrikák.....	37
3.4.6 Mentett állapot és statisztikák.....	39
4 Implementáció.....	40
4.1 Megosztott erőforrások.....	40
5 Próba hálók eredményei.....	42
6 Irodalomjegyzék.....	43
Függelék.....	45
6.1 Függelék.....	45
Nyilatkozat generatív mesterséges intelligencia alkalmazásáról.....	45

HALLGATÓI NYILATKOZAT

Alulírott **Csutár Márk Tibor**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot/ diplomatervet meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző, cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK nyilvánosan hozzáférhető elektronikus formában, a munka teljes szövegét pedig az egyetem belső hálózatán keresztül (vagy hitelesített felhasználók számára) közzétegye. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2026. 0X. XX

.....
Csutár Márk Tibor

Összefoglaló

Ezt majd a végén

Abstract

This too.

1 Bevezetés

Note: Legyen majd olyan fejezet, hogy dolgozat szerkezete

2 Irodalomkutatás és technológiák

Melanoma kiszűrésében és kezelésében az egészségügy is erősen támaszkodik a páciensek önellenőrzésére, amitől a gyanú felmerül. A laikus szem azonban szakavatatlan, nem feltétlen tudja mire kell figyelni. E dolgozatban célom, hogy a számítógép segítséget nyújtson ebben, és egy bőrelváltozásról készült fénykép alapján, hatékonyan és a lehető legpontosabban el tudja dönteni, hogy a képen melanoma vagy más bőrelváltozás látható-e.

A megvalósításhoz azonban nem mindegy, a számítástechnika mely technológiát használjuk. A következő fejezetben áttekintést szeretnék adni, milyen technológiák használhatóak erre a feladatra, tudományos háttérükről és hogyan érdemes gyakorlati használatba ültetni őket.

2.1 Klasszikus gépi képfelismerés

A gépi tanulás ma ismert fellendülése előtt is sok próbálkozás volt olyan programok írására, ami megmondja, mi szerepel egy képen. Működésük bemutatása nélkül, pár sikeres algoritmus, hogy kontextusba helyezhessük a ma elterjedt technológiákat:

- **Scale-invariant feature transform (1999):** Objektumok megtalálására működött jól, az objektum méret, forgatás, kisebb perspektíva változása mellett is. De kategorizálásra nem valamint magas a számítás igénye.
- **Histogram of oriented gradients (~1994):** Egyszerűbb, gyorsabb és hatékonyabb. Jól működött ha az objektumot az élei jól körülhatárolták; Például ember detektálásban pontos. Háttérzaj, az objektumok elrendezése, pózok, képtméret változására érzékeny.
- **Viola–Jones detector (2001):** Mérföldkőnek tekinthető. Nagyon gyors, alacsony az erőforrás igénye (valós időben is használható volt a korabeli hardveren). Egyszerűbb, jól struktúrált objektumok detektálásában, frontális arcdetektálásban jó. Nézőpont változására, takarásra azonban érzékeny volt.

Ezek a klasszikus gépi látási módszerek eredményeik ellenére korlátozottan bizonyultak. Bonyolult alakzatú objektumokat pontatlanul detektáltak, kézzel kellett megadni, hogy a képek részleteit hogyan érzékeljük, érzékenyek voltak zajra. Végképp nem értelmeztek szemantikát és kontextust a képeken. Ezekben sokkal jobbnak bizonyultak a mélytanuláson alapuló képfelismerő architektúrák. [1]

2.2 Kép előfeldolgozási technikák

Egy programnak, ami bemenetül egy képet kap és megmondja, mi van rajta, a legkritikább esetben ideális bemenet az eredeti nyers kép, ezért a bementi kép szinte minden esetben átmegy egy előfeldolgozáson. Előfeldolgozásra a számítási igény csökkentésén kívül azért is szükség lehet, hogy kiemeljük a számunkra releváns részleteket, elhanyagoljuk a szükségteleneket valamint a számokkal dolgozó gép számára feldolgozhatóbbra fordítsuk azokat. Ahogy látni fogjuk, a legtöbb képfelismerő technológia nem önmagában való, hanem alapul veszi képek előfeldolgozásának valamilyen kombinációját. [2]

2.2.1 Általános előfeldolgozási lépések

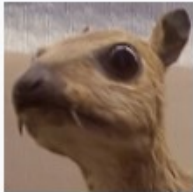
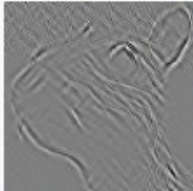
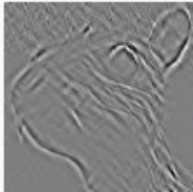
- **Átméretezés:** A neurális hálók fix méretű bemenetet várnak. Ez gyakran társulhat a kép kiegészítésével üres pixelekkkel, hogy ne veszítsünk információt skálázás okozta torzítással, vagy eltérő képarányú képeknél.
- **Szintér-átalakítás:** Ha a színek nem, csak az objektumok szerkezete fontos, elég a fekete-fehér. De ahol relevánsak a színek sem mindig az RGB a legjobb reprezentáció.
- **Normalizálás:** Ebben a lépésben a képeket egységes tartományba hozzák. Például 0-255 közötti RGB értékek helyett 0-1 közötti számra. Mivel a modell számokkal dolgozik ezzel a lépéssel stabilabb működés érhető el.
- **Adat kiegészítés:** Nem szigorúan előfeldolgozás. Ebben a lépésben mesterséges változtatásokat (kitakarás, forgatás) alkalmazunk a képekre. A tanító képeken lehet érdemes, hogy a modell jobban tudjon általánosítani.

2.2.2 Kernel konvolúció

A kernel konvolúció egy önállóan is hasznos technológia, mesterséges intelligenciáktól függetlenül, például képmanipulációra (elmosás, él kiemelés). Ereje abban rejlik, hogy a pixeleket a szomszédaikkal együtt vizsgáljuk. A részletek (él, sarok) pont a lokális kapcsolatokban, a képpontok szomszédaikhoz képesti megváltozásában rejlenek (szín, világosság megváltozása).

Működése a következő: A képen -amit szám mátrixként értelmezünk-, egy kisebb mátrixot végig csúsztatunk minden egyes képponton. Ez a kisebb mátrix a kernel. A kép minden pixelére a környező pixelek kernel értékekkel súlyozott összegét számoljuk ki.

A kernel értékektől függően nagyon sok oldalúan használható.

Operation	Kernel ω	Image result $g(x,y)$
Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Ridge or edge detection	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
Sharpen	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	

1. ábra Példák különböző kernel értékek eredményeire [4]

2.3 Neurális hálók képfelismerésre

Neurális hálók a gépi tanulás egy lehetséges megvalósításának családja sok más módszer közül. A módszerek között pár fő különbség, hogy milyen adatokon működnek jól és mennyire erőforrás igényesek. Egyszerű mintákra vannak sokkal hatékonyabb módszerek, a képfeldolgozásban azonban bonyolult mintázatokat kell felismerni.

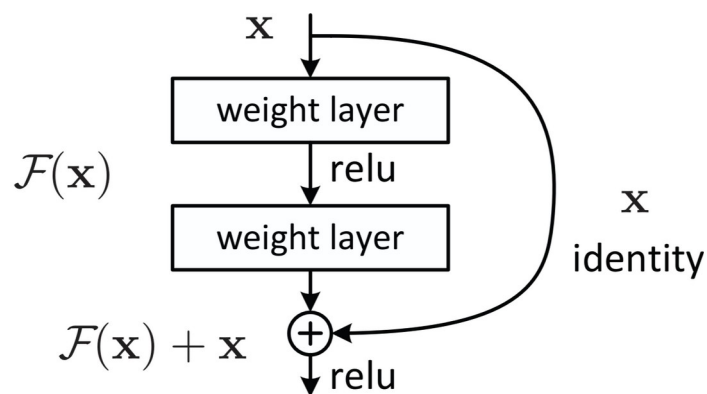
Nem is véletlen, hogy bár a klasszikus algoritmusok is tartalmaztak valamilyen gépi tanulást, azok a bonyolult mintákon korlátozottak voltak, így lényegében letarolták a képfeldolgozást a neurális hálók. További előnyük a klasszikus módszerekhez képes, hogy amíg azoknak kézzel kellett megadni, hogyan találják meg és dolgozzák fel a képek részleteit és az algoritmus végén volt egy osztályozó ami tanult; addig a neurális hálók elejüktől végükig, pusztán adaton, automatikusan tanulták meg, mely részletek relevánsak.

2.3.1 Konvolucionális Neurális Hálók

Angolul: Convolutional Neural Networks (továbbiakban: CNN). Motorjuk a már részletezett kernel konvolúció, azzal a különbséggel, hogy a kernel értékeit nem kézzel adjuk meg (az esetleges kezdeti értékeket kivéve), hanem a modell tanulja.

2.3.1.1 CNN-ek tipikus alkotó elemei:

- **Konvolucionális rétegek:** Ezek kernel konvolúciót végeznek, aminek eredménye egy olyan aktivációs réteg (feature map), ami olyan mintázatokot tartalmaz amiket a háló hasznosnak talál. Ilyen rétegekből több is előfordulhat, a modell elején az egyszerűbb, későbbi rétegekben komplexeb részletek észlelésére.
- **Pooling réteg:** Jellemzően a konvolúciós rétegeket követően. Mivel a detektált minták (él, sarok, stb.) az eredeti képhez képest sokkal kevesebb pixelen is értelmezhetőek. Pixelek egy csoportját egy beléjük táplált függvény alapján összevonja (pl.: max pooling esetén egy pixal ablak legnagyobb értékét adja tovább az új, 1db pixelnek). Alkalmazásukkal számítási teljesítmény és eltolásokkal szembeni robusztusság is nyerhető. Ugyanakkor nem minden architektúra használja, hasonló nyereség azzal is elérhető, ha kernel konvolúció során a kernelt 1 helyett több pixellel mozdítjuk odébb.
- **Előre csatolt kapcsolatok** (residual connections): Egy réteg kimenetét nemcsak az azt követő, hanem egy valahánnyal későbbi rétegnek közvetlenül is előre csatoljuk. Segíti, hogy az eredeti bemenet ne vesszen el teljesen, egyes minták később is hasznosak lehetnek. Mély hálók tanítását és használatát segíti.



2. ábra: Átugró kapcsolatok [7]

- **Osztályozó réteg:** Ez a gyakori elnevezés vagy fully-connected layer, de valójában azt csinálja, amire be van tanítva ami nem csak osztályozásban merül ki. Ez egy tipikus, neurális háló ami a kimenetet adja.

2.3.1.2 Néhány meghatározó CNN architektúra:

- Visual Geometry Group hálói (VGGNet): CNN architektúrák egy csoportja, VGG+rétegek száma elnevezési konvencióval (VGG16, VGG19). 2014-ben vált

ismerté az ImageNet Large Scale Visual Recognition Challenge-en (továbbiakban: ILSVRC) elért 92.7%-os pontosságával.

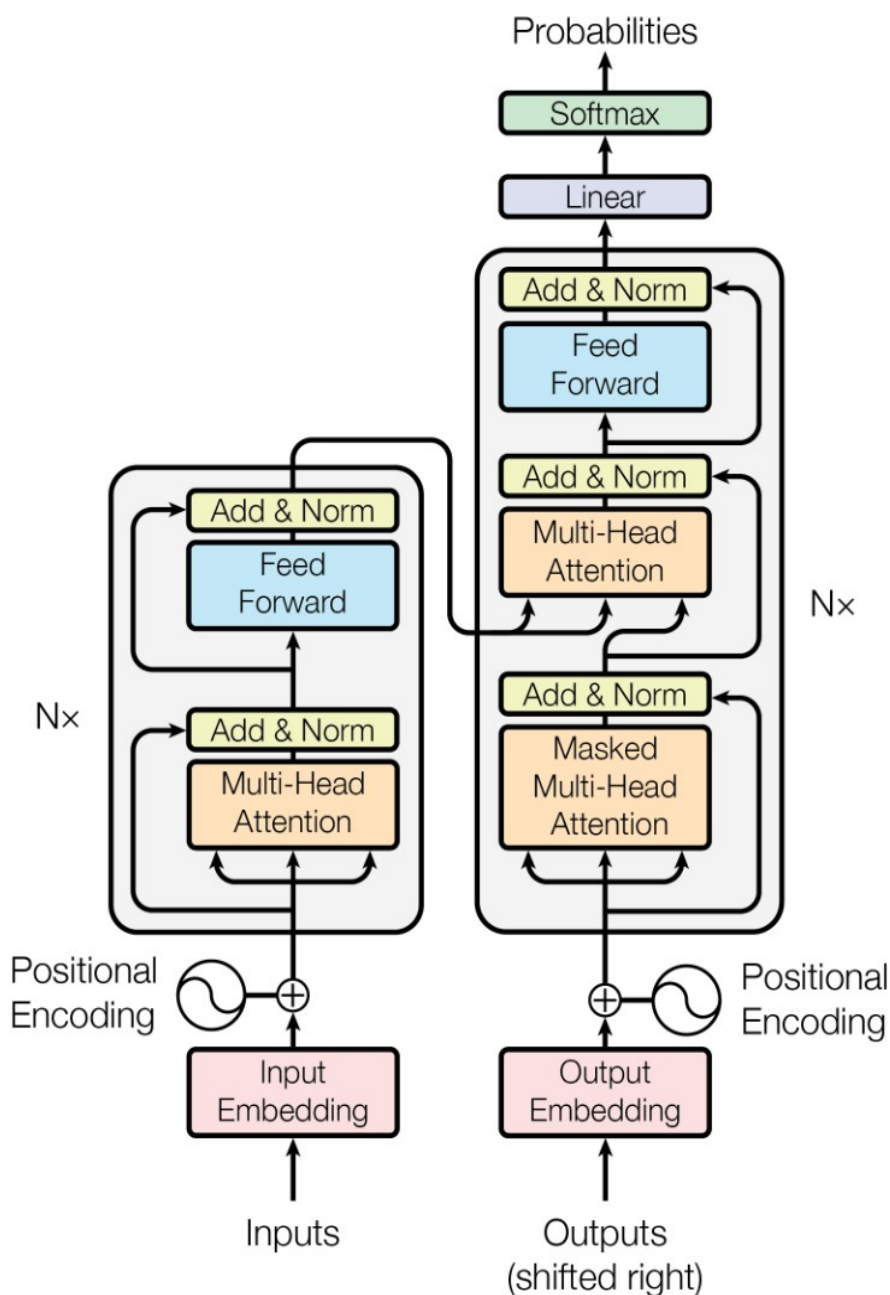
- Residual Networks (Előrecsatolt hálók, resnets): 2015-ben jelentek meg és meg is nyerték az az évi ILSVRC-t. VGG hálók a méretük növelésével már nem értek el pontosabb működést és nehezzé vált a tanításuk. A residual hálók előrecsatolt kapcsolatokat vezettek be, ezzel megkönnyítva a tanítást és (sokkal) nagyobb méretű hálók létrehozását, amik így nagyobb pontosságot tudtak elérni.
- ConvNeXt: 2022-ben megjelent modern, jelenleg egyik legkorszerűbb CNN architektúra. A transzformerek megjelenésével (lásd: Transzformerek képfelismerésre alfejezet) a CNN-ek háttérbe szorultak. A ConvNeXt megkísérli a CNN-ek versenyben tartását olyan, egyébként transzformerek által ihletett megoldásokkal mint: nagyobb kernel méret, réteg szintű normalizálás, előrecsatolt kapcsolatok. [8][9]

[3][7]

2.4 Transzformerek képfelismerésre

Transzformer modellek 2017-ben lettek bemutatva, először gépi fordításra használva, de az architektúra mögötti koncepciók generikusak annyira, hogy ma már számos és jelentős feladatra is épült transzformer alapú megoldás; Mint: Chatbot/szöveg generálás, audio és kép feldolgozás. Kép feldolgozásban a Vision Transformer (továbbiakban: ViT.) és az általa ihletett modellek ma az élen járók.

2.4.1 Eredeti transzformer architektúra



3. ábra: Eredeti transzformer architektúra az Attention is All you Need-ből [5]

Transzformerek kulcs újítása a **attention** mechanizmus, a bemenet egészen vizsgálja, hogy az egyes részek (tokenek) mennyire relevánsak a többi számára. Mondhatni, összebeszélnek a tokenek. Szöveges esetben egy szóra mennyire hatnak a korábbi szavak; például: „A cica aki egerészett gyors volt!” mondatban a „cica” és „gyors” szavak kapcsolódnak, még annak ellenére is, hogy a mondat 2 végén vannak. Ebben a cicás példában a gyors mint melléknév a cica egy tulajdonságát jelenti, de persze szavak máshogyan is hathatnak egymásra. Ilyen lehet (sok máson kívül) például nyelvtani kapcsolat (alany-állítmány, melléknév-főnév), névmások és hozzájuk tartozó entitások feloldása, idéző- és zárójelek összekapcsolása és jelentésbeli (szemantikai) kapcsolat.

Fontos kiemelni, hogy a transzformer modellek által feldolgozott alapegység a **token**. Szöveges esetben nem feltétlenül teljes szó, inkább szótő vagy toldalék. Egy ilyen szó-részt egy sok-sok dimenziós geometriai térbe ágyazással vektorra alakítanak. Képek esetében pedig 1-1 kép-szeletet (patchet) ágyaznak tokenbe.

Tokenizáció pontos folyamata szöveges esetben itt most nem releváns, képek esetén pedig Vision Transzformerek fejezetben fogom bemutatni, viszont a geometriai térbe egyezés miatt itt is fontos. Erre a reprezentációra azért van szükség, mert így a szavak (vagy amit beágyazunk) jelentése válik számíthatóvá. Egy klasszikus példa ezt jól szemlélteti: Képzeljük el, hogy a férfi, nő illetve király királynő szavakat ábrázoljuk irányvektorokkal, majd ebben az elképzeld koordináta rendszerben vonjunk ki a nőből a férfit. Ennek az eredménye egy új vektor, ami a férfi vektor végétől a nő vektor végébe mutat, ha ezt a vektort pedig hozzáadjuk a király vektorhoz, meg kell kapnunk a királynő vektort. Mintha ebben a geometriai térben az egyik irány nemiség jelentést hordozna.

A transzformer modellek azon része, ami a tokenek egymásra hatását számítja az ugynevezett **attention head** (3. ábrán: Multi-Head attention). Legintuitívabban úgy lehet elképzelni, egy attention head belső működését, mintha ez a kérdés szerepelne benne: „Ha itt ilyen token van, mely korábbi tokenek relevánsak és mennyire?”. Egy attention head arra, hogy mi alapján hatnak egymásra a tokenek, egyszerre csak 1 mintát tud megtanulni, ezért több attention head szerepel egy modellben (innen a Multi-Head).

Itt azt is fontos megjegyezni, hogy azt, hogy mi alapján értékeli 2 tokent összefüggőnek, azt a modell tanulja, ráadásul nulláról tanulja. Az említett összefüggőségi példák (nyelvtani, szemantikai, stb) feltűnése jellemző, de a tanulás során alakulnak ki, nem előzetes szándék alapján.

Az attention head-ek kimenete pedig lényegében egy módosítás az eredeti, bementi tokenen, ami a „jelentését” a kontextus ismeretében módosítja. Mivel ez a kimenet csak maga a módosítás, így a kimenetet előre csatoltan hozzá kell adni a bemeneti tokenhez (3. ábrán ilyen lépés az Add & Norm). Következő példán lehet ezt elképzelni: A fa szó más jelentéssel bír egy biológiai, mint egy számításelméleti könyvben, de a bementi szöveg tokenekre bontásakor viszont ugyanazt a vektort kapja. Az attention szándékozik kimenetétül adni azt az „eltolást” ami az eredeti „fa” vektort a szöveggörnyezet függvényében a „fa mint organizmus” vagy a „fa mint gráf” pozícióba tolja. Vagyis, a pusztán szó vektorát a kontextus alapján további jelentéssel gazdagítja.

Mivel több attention head szerepel egy modellben, amik különböző, de egyformán fontos részleteket tanulnak meg, a számításaik végén adott kimenetet össze kell fűzni. Ennek egy további óriási előnye, hogy emiatt az attention head-ek számításai párhuzamosan is végezhetők.

[5][8]

Attention számítása:

1. Bemenet: E tokenek
2. Legyen 3 mátrixunk, Query, Key, Value: W^Q, W^K, W^V ! A mátrixok értékei egyben a modell paraméterei, amik a tanulás során változnak, finomhangolódnak.
3. Mindegyik mátrixot szorozzunk meg mindegyik E_i bementi tokennek, így állítva elő q, k, v vektorokat.

$$q_i = W^Q E_i \quad k_i = W^K E_i \quad v_i = W^V E_i$$

4. Vegyük mindegyik q vektornak mindegyik k vektorral a skaláris szorzatát. Legyen ez az attention score.

$$\text{score}_{ij} = q_i \cdot k_j$$

A skaláris szorzat értelmezhető egy vektor másikra vetett vetületeként, ami akkor nagy, ha a vektorok hasonló irányba néznek. Ez a hasonló irányba nézés, illeszkedés, felel meg a 2 token kapcsolódásának, hogy mennyire „attendálnak” egymásra.

5. Az attention score-kat normalizáljuk úgy, hogy minden oszlopa megfeleltethető legyen egy eloszlásnak, softmax képlet alkalmazásával, aminek tulajdonsága, hogy az elemek oszlop összege 1

$$\alpha_{ij} = (\exp(\text{score}_{ij})) / (\sum_j \exp(\text{score}_{ij}))$$

Ekkor minden α érték 0-1 közötti valós szám és a belőlük alkotott mátrixot nevezhetjük **attention pattern**-nek.

6. Az **attention pattern** minden i-ik sorának minden elemét össze skalárszorozzuk az i-ik value vektorral.

$$z_{ij} = \alpha_{ij} v_j$$

Ezek a z értékek már nem valós számok, hanem a value vektorok lényegében súlyozva az attention pattern értékeivel. Vagyis a value vektorok súlyozva azzal, hogy a 2 token mennyire attendál egymásra.

7. Végeredményben a súlyozott value vektorok mátrixának oszlopait összegezzük.

$$\Delta E_i = \sum_j z_{ij}$$

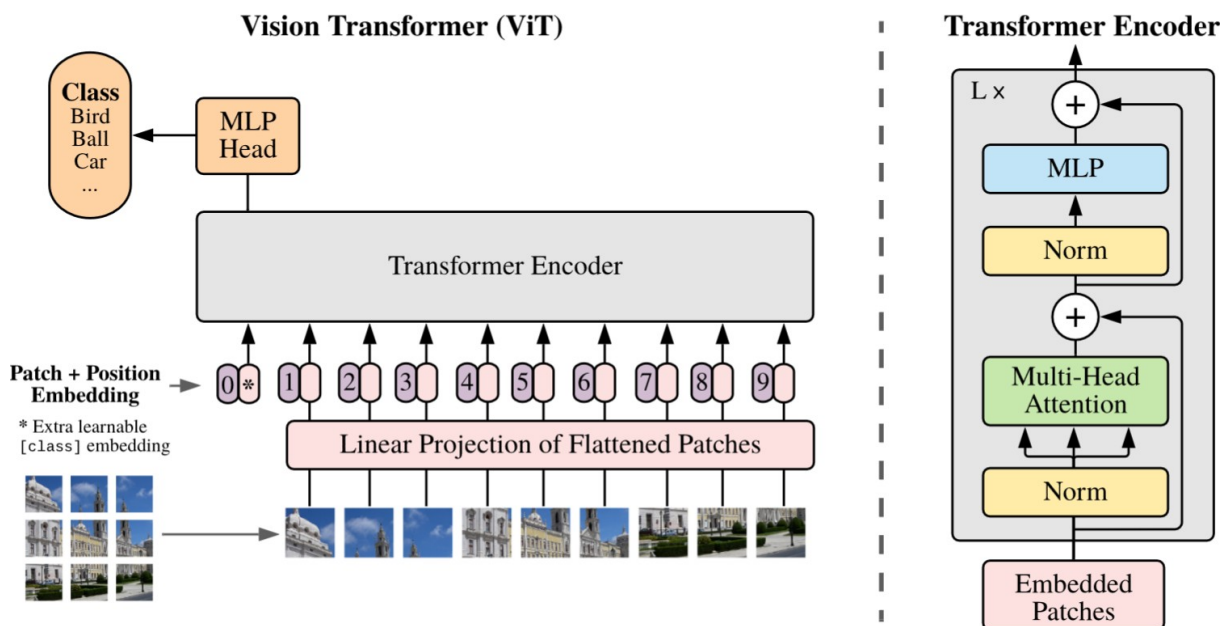
Az így kapott ΔE_i tartalmazza azt a módosítást, amit az eredeti i-ik E_i tokenhez adva, az annak jelentését a kontextus függvényében gazdagítja.

A fenti folyamatot a következőképpen foglalnám össze és fordítanám kicsivel emberibb nyelvre: 2 token illeszkedését, egymásra hatását a skaláris szorzás adja, viszont a skaláris szorzás operandusai nem közvetlenül a bemeneti tokenek, hanem azok szorzata a Query és Key mátrixokkal, mely mátrixok a modell paraméterei, ezáltal tanulja meg a modell, hogy összefüggő szavak esetén a mátrixok ugyanezekkel tokenekkel szorozva egymásra illeszkedő vektorokat kell adjanak. Miután pedig megvan, hogy mely token van hatással melyikre, a Value mátrixot a ható tokennel szorozva és súlyozva kapjuk meg azt a módosítást, amit hozzá kell adni a hatott tokenhez. És mivel a Value mátrix is a modell paramétereit tartalmazza, az, hogy egy token milyen módosításnak felel meg egy későbbin, azt a modell szintén tanulja.

Pongyola példa: „zöld erdő” szavakkal $q=\text{Query}*\text{erdő}$ és $k=\text{Key}*\text{zöld}$ után $q \cdot k$ megmondja, hogy „zöld” mennyire hat „erdő”-re, ennek ismeretében pedig $v=\text{Value}*\text{zöld}$ egy „zöldítő” vektort ad, amit ha az „erdő” tokenjéhez adnánk, akkor az gazdagodna azzal a jelentéssel, hogy egy élő nyári zöld erdőre utal, nem pedig egy lombhullatott télire.

[8]

2.4.2 Vision Transformer



4. ábra: Vision Transformer modell architektúra [6]

Szöveges példák után pedig nézzünk a képekkel dolgozó ViT motorházteteje alá. A 4. ábrán látható architektúra elemeit részletezve.

- I. **Bemeneti kép patch-ekre bontása:** A kép felszeletelése, $n \times n$ -es részekre. Ezeknek a képszeleteknek a mérete és ebből következően, hogy hány patch-re bomlik a kép céltól és erőforrások fényében változhat. Az eredeti cikkben 16×16 -os patcheket használtak.
- II. **Embedding:** Egy rgb kép esetén egy patch $n \times n$ pixel $\times 3$ szín csatorna. Kilapítással vektorra alakítjuk (pl.: 16×16 pixel $\times 3$ szín = 768 dimenziós vektor). Ezután hozzárendeljük, hogy a teljes képen hol helyezkedik el a patch. Erre azért van szükség, mert a párhuzamosított attention head-ek és az utánuk végzett összegűzés érzéketlen a patch-ek pozíciójára, így a modell későbbi részei számára elveszne, hogy amit vizsgál képrészlet például közepén vagy sarokban van. Ez a patch és pozíció vektor együttesen a token.
- III. **Transformer encoder:** Kompozit blokk. A bemeneti beágyazott tokenek az alábbi lépéseken mennek keresztül.
 1. Tokenek normalizálása.
 2. Normalizált tokenek áteresztése egy több attention head-ből álló self attention rétegen.

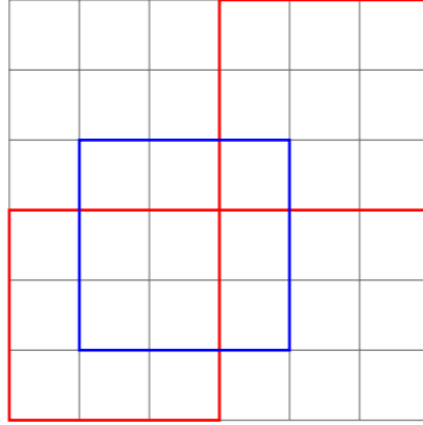
3. Az attention head-ek kimeneteinek és az előrecsatolt bemeneti tokenek összeadása.
4. Már self attention-t tartalmazó tokenek újbóli normalizálása.
5. Többrétegű perceptron (Multi-Layer Perceptron, MLP) réteg. Ez valójában egy sima, általában több rétegű neurális háló, hasonlóan a CNN-ek végén szereplő fully-connected réteghez. Ez a réteg tokenenként dolgozik, vagyis a tokenek itt már nem beszélnek össze, mint az attention-nél. Miután az attention gazdagította a tokeneket, hogyan kapcsolódnak egymáshoz. Ez a blokk absztraktabb mintákat tanul meg az egyes tokeneken. Egy cicás kép példáján az attention megtanulja, hogy az orr, fül, szem patch-ek kapcsolódnak és hogy egy arc részei; az MLP pedig pl. önállóan az orr patch-en a szín, forma, arc kontextus ismeretében kimenetül adja, hogy ez a patch valószínűleg egy cica orr.

IV. **Osztályozó:** Végül a transformer encoder kimenetét egy osztályozónak adjuk. Ez egy kis, legtöbbször 1 rétegű neurális háló, ami már a kívánt osztálycímkeket (vagy amire be van tanítva) adja kimenetül.

[6]

2.4.3 Swin Transzformer:

A részletezett Vision Transzformerre lehet úgy gondolni, mint az eredeti vision transzformerre. Azonban mivel ez a modell lényegében 1-1 átvétele az eredetileg szövegekre kitalált transzformer architektúrának, annak nagy számítási igényével, azóta jelentek meg további, optimalizáltabb ViT-ek, még inkább képek feldolgozására specializálódva. Ilyen például a Data-efficient Image Transformer (DeiT, 2021) vagy a Multi-Axis Vision Transformer (MaxViT, 2022). Az én feladatomhoz a Swin Transzformert találtam érdekesnek, ezt szeretném röviden ismertetni.

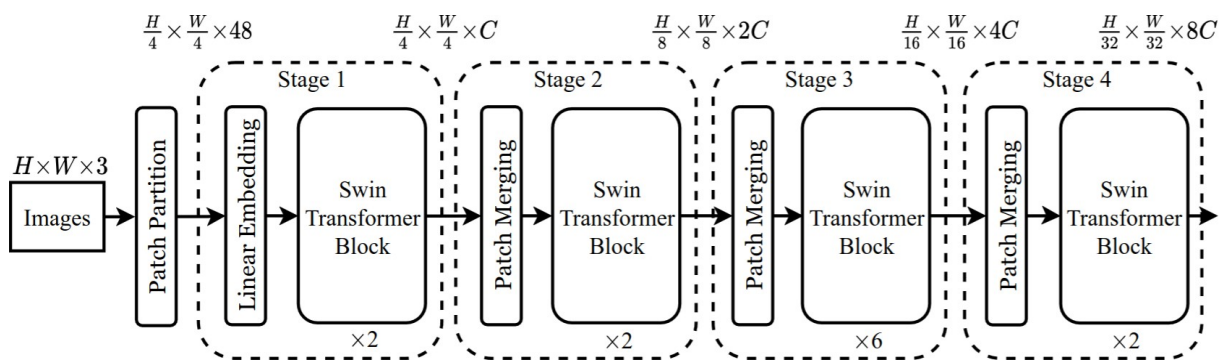


5. ábra: "Lokális ablakok, eltolással rétegenként" [8]

Az eredeti ViT-ben minden token (patch) figyel mindegyik másikkra, emiatt a képméret növekedésével a számítási igény négyzetesen nő. A Swin Transformer ennek mérséklésére vezette be a csúsztatott ablakos (shifted window) technikát, hogy a képmérethez képest lineáris komplexitás növekedést érjen el.

Csúsztatott ablak esetén nem minden tokenre minden másikkal számít összefüggést, hanem csak 1, fix méretű ablakon belül beszélnek össze a tokenek és a képet több ilyen ablakkal fedi le. Ezzel azonban a képrészletek csak lokálisan, az ablakon belül figyelnek egymásra, ezért későbbi rétegekben elcsúsztatva, szándékosan a korábbi ablakokkal átfedő részekre is illeszt ablakot, így nyerve globális kontextust.

[8][11]

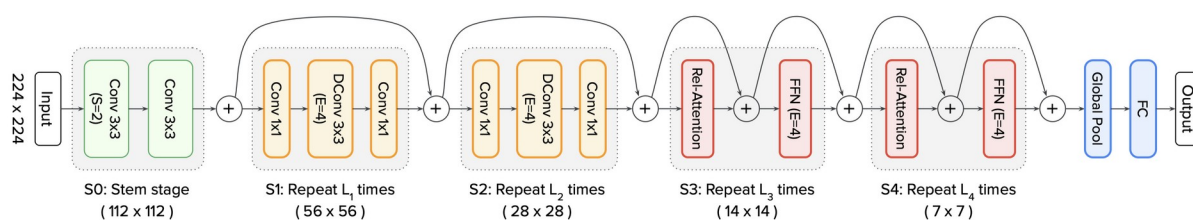


6. ábra: Swin Transformer architektúra [11]

2.5 Hibrid architektúrák

CNN-ek nagyon jól kezelik a lokális mintákat, kevés adaton is jól működnek, alacsony a komplexitásuk, viszont globális összefüggések megtanulására csak sok réteggel képesek. Transzformerek ezzel szemben azonnal az egész kép kontextusát figyelembe veszik, viszont alaptól nem rendelkeznek lokális minta felismeréssel, ezért ezt is nulláról kell megtanulniuk ami következményeként rengeteg adat szükséges a tanításukhoz (adatéhesek) és a számítási igényük is nagyra tud nőni.

A két megközelítést kombinálni és így az előnyeik egygyütes megnyerését kísérik meg a hibrid hálók. Számos ilyen modell létezik már (példul: Conformer, MobileViT), ilyenek közül a Convolution + Attention hálót (CoAtNet) szeretném részletezni, ami nem az első próbálkozás a két technológia egyesítésére, viszont átfogóan kísérli meg annak helyes végrehajtását.



7. ábra: CoAtNet architektúra felépítése [10]

Az 7-ik ábrán látható CoAtNet háló az öt bemutatott cikk-ben szemléltetett felépítése 2 konvolúciós és 2 attention réteggel (C-C-T-T), de a cikk számításba vesz és kísérletezik ettől eltérő konvolúció/attention aránnyal. Fontos, hogy a konvolúciót és az attention-t nem keveri, hanem először használ konvolúciót az alacsony szintű minták megtanulására és később attentiont a kép szintű összefüggések megtanulására.

A rétegek működése részletesebben:

- S0: Bemenet előfeldolgozása, egyszerű konvolúcióval.
- S1-2: Mobile Inverted Bottleneck Covolution (MBConv) blokkok. Erőforrásszegény eszközökre tervezett CNN-ekből (EfficientNet, MobileNetV2) átvett ötlet. Először növeli a csatorna számot (1 csatorna 1 kernel kimenete, valamilyen alacsony szintű minta), majd minden csatornán külön futtat 1-1 szűrőt (Depthwise Convolution, Dconv), majd visszacsökkenti a csatornaszámot, hogy megegyezzen a bemenet dimenziójával és előrecsatoltan összegezni lehessen vele.

- S3-4: Attention blokkok, megtanulja a kép egészére a képrészek közötti kapcsolatokat. Fontos különbség eredeti ViT-het képest, hogy nem abszolút pozíciót rendel hozzá a patch-ekhez, hanem relatívat (pl.: ettől-attól a patchtől balra/jobbra van ez a patch), hogy koordináták helyett kapcsolatokat tanuljon és így eltolásokra ne legyen érzékeny.
- Kimeneti réteg, átlagolós pooling-al és egy egyszerű neurális hálóval.

[10]

3 Tervezés

Gépi tanulás sem kivétel a megvalósítást megelőző tervezés igénye alól. A következő fejezetben részletezem, a melanoma detektorom megvalósításáig milyen lépéseken keresztül jutok el, ahhoz milyen erőforrásokat és technológiákat használtam és -ahol ez releváns- azokat milyen paraméterekkel, valamint, hogy az általam elkészítendő megoldásoknak milyen követelményeknek kell megfelelniük.

Bár a végső cél egy melanoma detektor és az ahhoz vezető utat fogom részletezni, azt előre kiemelem, hogy ebben a fejezetben még nem a végcélnak megfelelő neurális hálót tervezem meg. Helyette a modern képfelismerő architektúrák teljesítményének tesztelését bőrbetegségek osztályozásában. Emelett azért döntöttem, hogy mérés alapján tudjam a legmegfelelőbb architektúrát választani a végleges neurális háló megvalósítására.

3.1 Technológiai háttér

A dolgozatomban tervezett neurális hálók létrehozásában milyen technológiákra, szoftverekre és keretrendszerekre támaszkodok.

3.1.1 Számítási platform

Felhő alapú fejlesztői környezetekkel való korábbi tapasztalataim során szembesültem azzal, hogy bár kényelmes a függőségek telepítésének terhétől megszabadulni, a felhasználás mértékének korlátozása miatt könnyű kifutni a számítási keretből még az ezen dolgozatban érintettnél jóval kisebb hálók esetén is. Ez indokolja, hogy az orvosi adatokat osztályozó hálókat saját eszközön futassam és tanítsam.

Azonban neurális hálók tanítása rengeteg számítást jelent a számítógépen. Hogy a nagy számú számításokat a grafikus kártyámmal (továbbiakban: GPU) tudjam elvégeztetni, az NVIDIA által fejlesztett Compute Unified Device Architecture (CUDA) platformot fogom használni.

3.1.2 Programozási nyelv és keretrendszerek

Ebben az alfejezetben bemutatom, mely programozási nyelvet és mely ahhoz implementált kiegészítő könyvtárakat és keretrendszereket használtam fel a céloom elérésére.

3.1.2.1 Python

A feladatom leginkább számítás igényes része a neurális hálók betanítása, melyet pont ezért cuda-n keresztül GPU-mra szervezek ki. A további részfeladatok pedig már nem erőforrás kritikusak, cserébe nagyban megkönnyíti ezek elvégzését, ha a választott nyelv kényelmesen használható. Ezen adottságok miatt döntöttem, a data science és mesterséges intelligenciával foglalkozók körében egyébként és bevált és kedvelt Python programozási nyelv mellett, melynek gazdag az eszköztára és érhető el rá kifejezetten neurális hálók fejlesztésére kifejlesztett keretrendszerek.

3.1.2.2 Pytorch

A pytorch egy Meta által fejlesztett, nyílt forráskódú python könyvtár, amit neurális hálók fejlesztésére hoztak létre. Kifejezetten támogatja a számítások GPU-n történő elvégzését, rugalmas, kényelmesen használható, könnyen tanulható. Iparban is használt, de a könnyű tanulhatósága miatt kutatók körében egyel népszerűbb, szemben az alternatív TensorFlow keretrendszerrel, ami nehezebben tanulható, de az iparban erősebb. Saját neurális hálók létrehozához biztosított eszköztáron kívül arra is ad lehetőséget, hogy kész hálókat plug-n-play módon töltsünk be.

Képfeldolgozásra említésre érdemes könyvtárak még a PyTorchot kiegészítő torchvision és timm könyvtárak. Torchvision kiegészítő eszköztárat ad, ide értve a képeken végezhető transzformációkat, amik előfeldolgozáshoz jelentősek, továbbá tartalmazza pár neves architektúra definícióját. Timm pedig egy átfogó mélytanulási könyvtár, mely a modellek betöltésétől, a tanításig és validációig is tartalmaz megvalósításokat, de az én dolgozatomhoz a torchvision által nem tartalmazott modellek betöltését megkönnyítő funkciója releváns.

3.1.2.3 Pandas

A pandas egy nyílt forráskódú adatelemző python könyvtár. Úgy is lehet rá gondolni mint egy táblázatkezelőre, ami grafikus felület helyett pythonból használható. A megvalósítás során statisztikák gyűjtésére, kezelésére és kiértékelésének megkönnyítése miatt döntöttem a használata mellett.

3.1.2.4 Captum

Neurális hálók méretének növekedésével egyre kevésbé átlátható, hogy a bemenet mely részletei vezetnek adott kiemeléshez, vagyis egyre nehezebb belelátni a hálóba, hogy mit

tanul meg, mely részleteket tartja fontosnak? Ebben nyújt segítséget a Captum, ami egy nyílt forráskódú, PyTorch-ra épített könyvtár, modell értelmezést segítő eszköztárral.

3.1.3 Egyéb szoftverek és technológiák

- **Jupyter notebooks:** Neurális hálók betanítása, bár programozottan történik, egy szotverprojekthez képest a részprogramok létrehozása, azok futtatása, eredményeiknek kiértékelése nagyobb rugalmasságot és betekintést igényel. Erre remek eszköz egy jupyter notebook, ahol egyetlen füzetben több részprogramot lehet megírni, külön-külön is futtatni és jegyzetekkel kiegészíteni.
- **Visual Studio Code:** A Microsoft népszerű, nyílt forráskódú kód szerkesztője, rengetek produktivitást segítő kiegészítővel, köztük a lehetőséggel hogy Jupyter notebook-okat kezeljen.
- **Git és GitHub:** A fejlesztés folyamatában verziókezelésre, eszközök közötti kényelmes szinkronizációra, a kódbázis megosztására.
- **LibreOffice Calc táblázatkezelő:** A dolgozat során felmerülő elemzési feladatokra, különösen az adathalmazok elemzésére.

3.2 Adathalmaz és elemzése

A dolgozatomban olyan osztályozó(ka)t szándákozok létrehozni, amely bőrelváltozásokról készült kép bemenet alapján megállapítja azok fajtáját. A technológia már részletezett jelenlegi állása és a feladat karakterisztikából következően neurális háló/transzformer architektúrák használatával érdemes ezt megvalósítanom. Mivel a neurális hálók adaton tanulnak -szemben egy tipikus szoftverrel, aminek a belső működését, annak paramétereit a programozó kézzel adja meg-, az osztályozó végső pontosságában kulcs szerepet játszik a megfelelően megválasztott tanító adathalmaz.

A bőrelváltozások osztályozásánál kiemelten arra szeretnék fókuszálni, hogy az melanóma-e vagy sem. Ebben az esetben egy igen/nem válasz, vagy százalékos melanoma gyanúság is elég, de az idáig vezető úton több kategória használata és a modellek azon való tanítása és kiértékelése számomra hasznos következtetésekre vezethet a fejlesztéssel kapcsolatban, valamint maga a modell is megtanul több részletre figyelni. Adathalmaz szempontjából fontos, hogy az egyes kategóriákba tartozó képek egymáshoz képest megfelelő arányban jelen legyenek.

Fontos szempont továbbá az adatok mennyisége. Transzformer típusú hálónál, ha nulláról tanulnak milliónyira, de előtanított modellek esetén is ezres nagyságrendű képre van szükség.

Online, ingyenesen elérhető adathalmazok közül, a szempontjaimnak a Melanoma Detection Dataset [12] (továbbiakban: MDD), illetve a Skin Lesions Classification Dataset [13] (továbbiakban: SLCD) bizonyult ígéretesnek. Mindkét adathalmaz valójában csak célspecifikusan előre összeválogatott részhalmaz a az International Skin Imaging Collaboration (ISIC) képeinek, illetve az SLCD még további forrásokból is tartalmaz képeket.

Annak kiderítésére, hogy végeredményben melyik adathalmaz a legmegfelelőbb a számomra, részletes adatelemzést végzek.

3.2.1 Vizsgálati módszerek

- Vizuális ellenőrzések: Szemrevételezéssel végigfutom a képeket, figyelve a képek minőségére, elmosódásokra, megvilágítottságára, esetleges duplikált képekre. és kiszűrni kiugró, irreleváns, vagy nem megfelelően (pl. túl közlelől/válolelől) fotóztott képeket. Itt azt is érdekes megvizsgálni, hogy a képeken hol helyezkednek el a modellnek releváns részletek (kép közepén vagy eltérő helyeken). Ez a lépés azért fontos, hogy meggyőződjek, a modell tényleg a célnak releváns részleteket tanuljon de előfeldolgozási lépésekhez is hasznos ismeretek szerezhethők.
- Adathalmazok statisztikái: Ebben a lépésben megvizsgálom a képek méretét, fájlformátumokat, a képek, osztályok számát, azok eloszlását a részhalmazokban, ezáltal az adathalmaz kiegyensúlyozottságát. A képméreteket, képarányok ismerete segíthet eldönteni, hogy előfeldolgozásnál hogyan hozzam egységes dimenzióra a képeket. Kiegyensúlyozottság ismeretére azért van szükség, mert kiegyensúlyozatlan adathalmaz esetén, a modell úgy is nagy százalékos pontosságot érhet el akkor is, ha a kis mintaszámú osztályokat egyszerűen csak ignorálja, valamint azt is segít eldönteni, szükség van-e adat augmentációra. A tanító, értékelő és teszt részhalmazok eltérő eloszlása esetén pedig a modell más eloszlást tanul, mint amin tesztelve van és ez pontatlan kiértékelést végeredményben pedig a modell pontatlan működéséhez vezethet.
- Osztály integritás: Azt vizsgálom, van-e átfedés a tanító és teszt képek között? Átfedésnek az lehet a hatása, hogy a modell kevésbé általánosít, hanem inkább konkrét példákat memorizál.

- Pixelstatisztikák: RGB képek csatornánkénti átlagát és szórását számítom ki. Ennek eredményei a modell bemenet megfelelő normalizáláshoz használhatóak.

3.2.2 Eredmények

Általános statisztikák

- MDD osztályai: *melanoma, nevus, seborrheic keratosis*.
- SLCD osztályai: *Actinic keratoses, Basal cell carcinoma, Benign keratosis-like lesions, Chickenpox, Cowpox, Dermatofibroma, Healthy, HFMD, Measles, Melanocytic nevi, Melanoma, Monkeypox, Squamous cell carcinoma, Vascular lesions*

Dataset	Osztályok száma	Képek száma	Felbontás és képarány	Formátum
Melanoma Detection Dataset	3	2750	változó	.jpg
Skin Lesions Classification Dataset	14	36656	változó	.jpg

1. táblázat: Adathalmazok általános statisztikái

Vizuális észrevételek

MDD esetén a képeknek jó a minősége, megvilágítottak, fókuszáltak, és az elváltozások többnyire középre igazítottak, pontosan a releváns részek látszódnak, viszont gyakori a méretük miatt a képek széléig terjeszkedő bőrelváltozások. Szintén gyakoriak még a „távcsőszerű” (valószínűleg dermatoszkópon keresztüli) képek, ahol az elváltozás körül, köralakban a kép szélei feketék. Ez akár probléma is lehet, ha a modell ezt releváns részletként tanulja meg.

SLCD esetén bizonyos osztályok képei nagyon eltérőek a többitől: *Chickenpox, Cowpox, Healthy, HFMD, Measles, Monkeypox*. Ezekben a képek gyakran távolabbról készültek, több testrészt is tartalmaznak, valamint sok bennük a duplikált, előre augmentált kép. A többi osztályban a képeknek jó a minősége, megvilágítottak, fókuszáltak, és az elváltozások középre igazítottak, pontosan a releváns részek látszódnak. A képek széléig nyúló és dermatoszkópon keresztüli képek ebben az adathalmazban is gyakoriak.

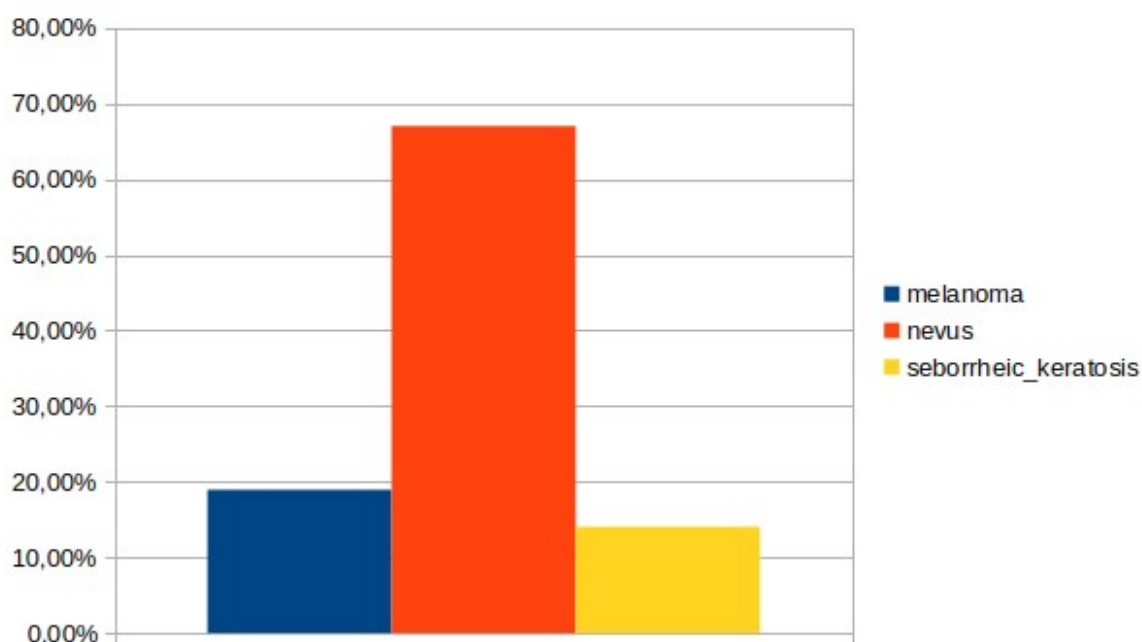
MDD elemzése

Képek felbontása változó, 576 x 540-tól, 6748 x 4499-ig, és a kettő között még sok mással, összesen 209 különböző felbontással. Leggyakoribb felbontások: 4288x2848 (23%), 1024x768 (21%), 3008x2000 (17%). Képarányokból pedig 200 különböző létezik, leggyakrabbak: 4:3 (23%), 134:89 (23%), 188:125 (17%). Egészen kaotikus.

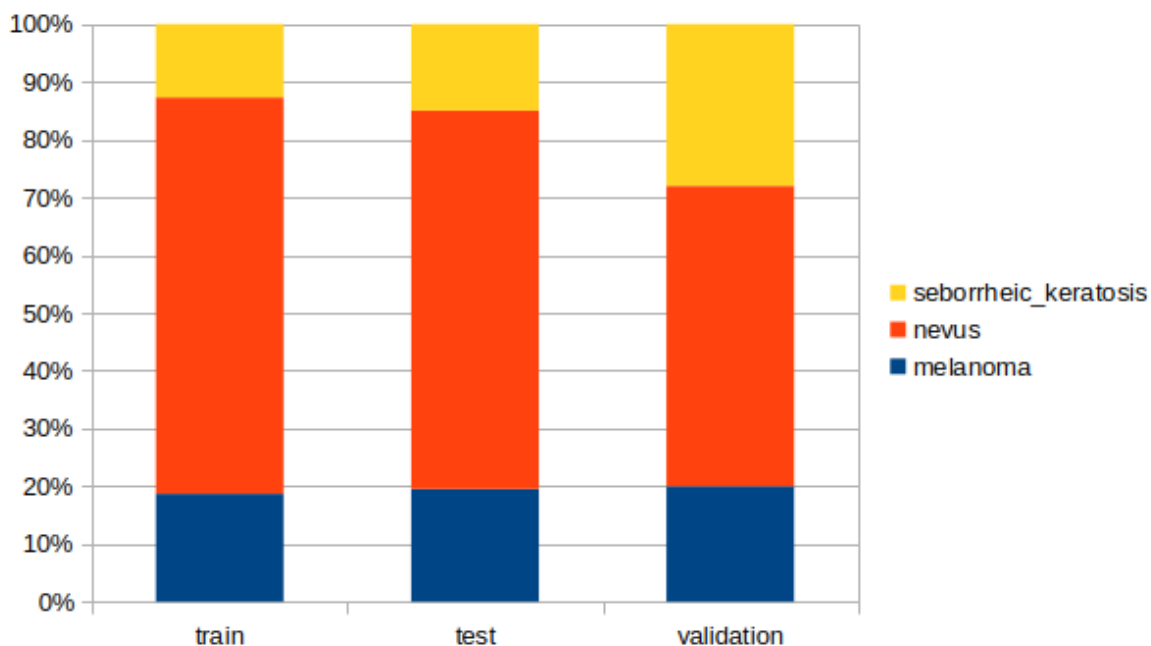
Átlag felbontás	Felbontások szórása	Normalizált RGB színcsatornák átgala	Normalizált RGB színcsatornák szórása
3281.8 x 2228.6	1882.7 x 1214.5	(0.711, 0.548, 0.505)	(0.169, 0.170, 0.185)

2. táblázat: MDD pixel statisztikái

Összes kép között az osztályok ~19-67-14% arányban oszlanak meg (8. ábra). Ez mérsékelten kiegyensúlyozatlan, a modell már hajlamos lehet a többségi osztály felé húzni. Felhasználási részhalmazonként a train és test képek szinte egyformán oszlanak meg, validation esetén tolódik el a „nevus” és „seborrheic keratosis” osztályok aránya. Átfedés a halmazok között nincs.



8. ábra: Osztályonkénti összes képek aránya MDD adathalmazban.



9. ábra: Osztályok eloszlása felhasználási részhalmazonként belül az MDD adathalmazban

SLCD elemzése

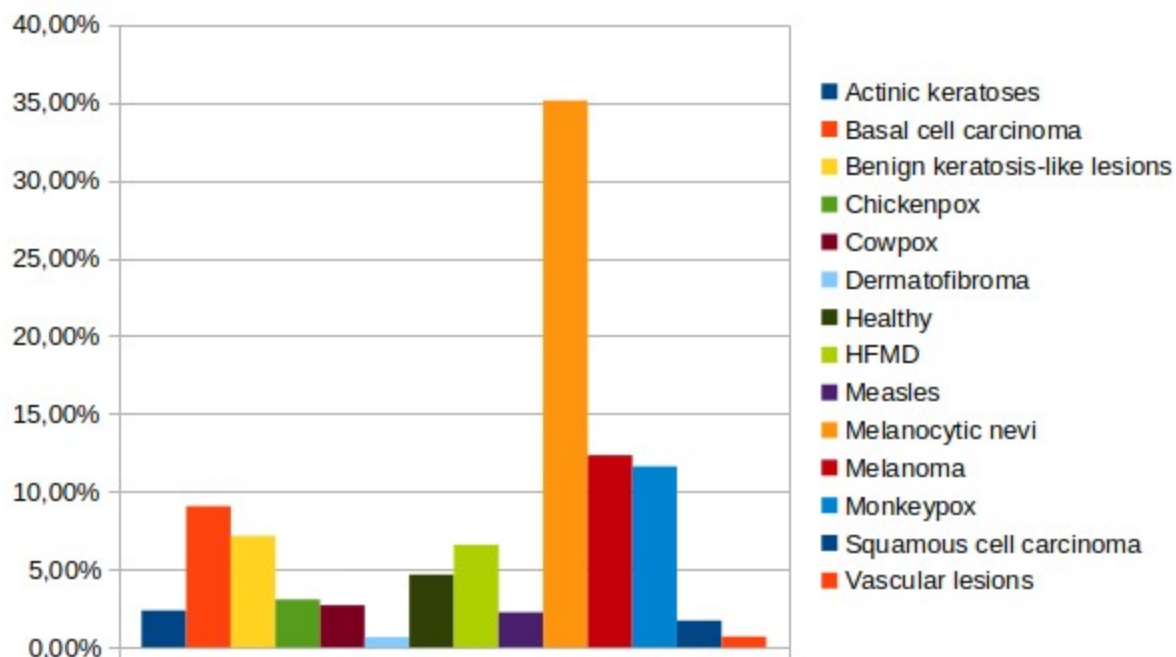
Képek felbontása változó, 69 x 69-től, 2448 x 2448-ig, összesen 305 különböző felbontással. Viszont egymáshoz képest jóval kevésbé eltérőek és osztályokon belül már jellemzőek az egységes felbontások. A leggyakoribb felbontások: 1024x1024 (34%), 600x450 (27%), 224x224 (18%). Képarányokból pedig 106 különböző létezik, leggyakrabbak: 1:1 (négyzetes, 64%), 4:3 (29%). Vagyis az összes kép 3 legnagyobb hányada legalább önmagukban egységes felbontású, képarányokból pedig még valamivel jobb a helyzet.

Átlag felbontás	Felbontások szórása	Normalizált RGB színcsatornák átgala	Normalizált RGB színcsatornák szórása
674.3 x 609.2	334.5 x 333.1	(0.675 , 0.536, 0.503)	(0.207, 0.195, 0.205)

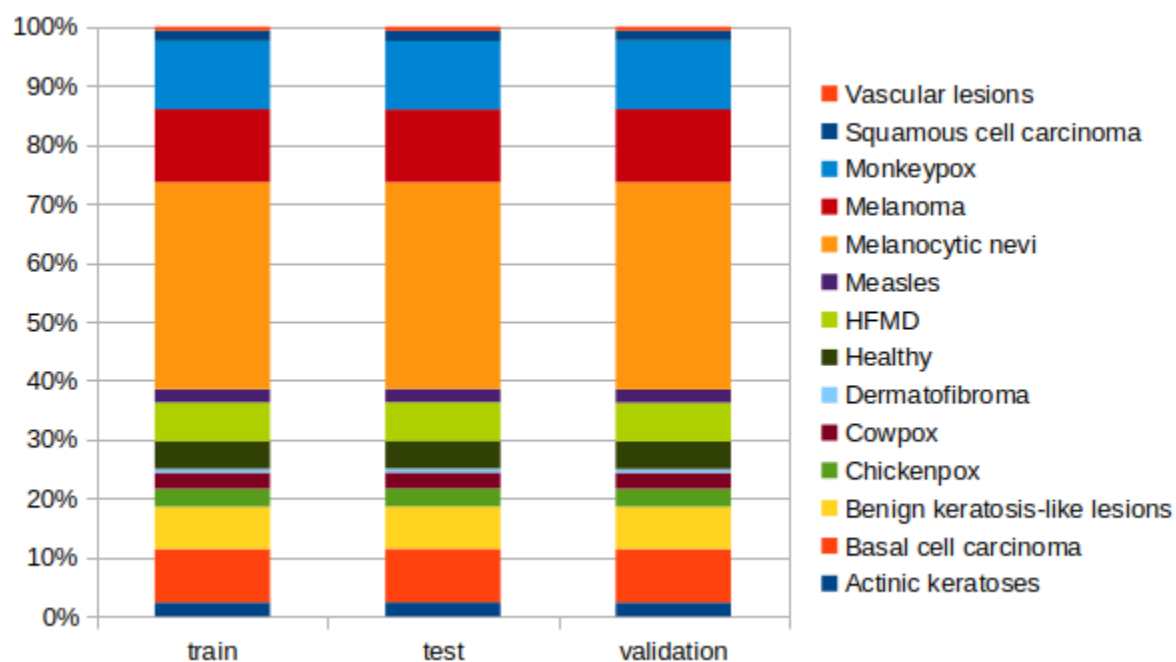
3. táblázat: SLCD pixel statisztikái

Összes kép között az osztályok aránya erősen kiegyensúlyozatlan, legnagyobb mintaszámú 35%, legkisebb: 0.69% (10. ábra). Ezt biztosan ellensúlyozni kell tanításkor, ha legalább mérsékelni akarom a többségi osztályok felé húzást.

Minden részhalmazon belül szinte azonos ($\pm 0.1\%$) az osztályok aránya (11. ábra). Átfedés a részhalmazok között nincs.



10. ábra: Osztályonkénti összes képek aránya SLCD adathalmazban.



11. ábra: Osztályok eloszlása felhasználási részhalmazonként az SLCD adathalmazban

3.2.3 Módosított SLCD adathalmaz

Mindkét adathalmaz hagy maga után kívánni valót. Mindkettő kiegyensúlyozatlan, a felbontások változóak. Összességében viszont az SLCD bizonyul használhatóbbnak, abban ugyanis a problémák legfőbb oka a kiugró osztályok, így ezen osztályok elhagyásával a

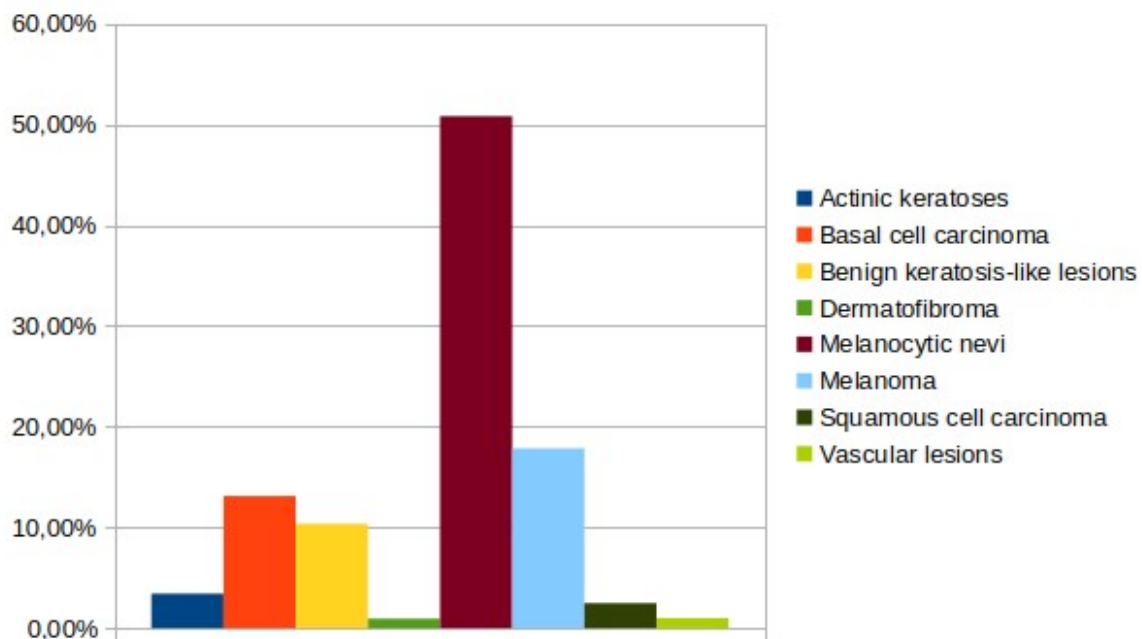
problémák egy része is megszűnik. A többi osztályban a képfelbontások és képarányok is homogénebbek, a releváns képrészletek pedig jó minőségben, a képek közepén szerepelnek. Dermatoszkópiás képek továbbra is előfordulnak, ugyanakkor a maradék osztályok mindegyikében jelen vannak, így ha a modell ezt el is kezdené releváns részletnek gondolni, ez mégsem különbözteti meg egyik osztályt sem. Ráadásul MDD-hez képest a képek mennyisége is sokszoros, sőt, mivel SLCD-nek is ISIC az elsődleges forrása, valójában SLCD magában foglalja a(z) MDD képeit is. A kiegyensúlyozatlanságot viszont továbbra is ellensúlyozni kell a tanítás során.

Chickenpox, Cowpox, Healthy, HFMD, Measles, Monkeypox osztályok elhagyása után a módosított SLCD adathalmaz jellemzői a következők:

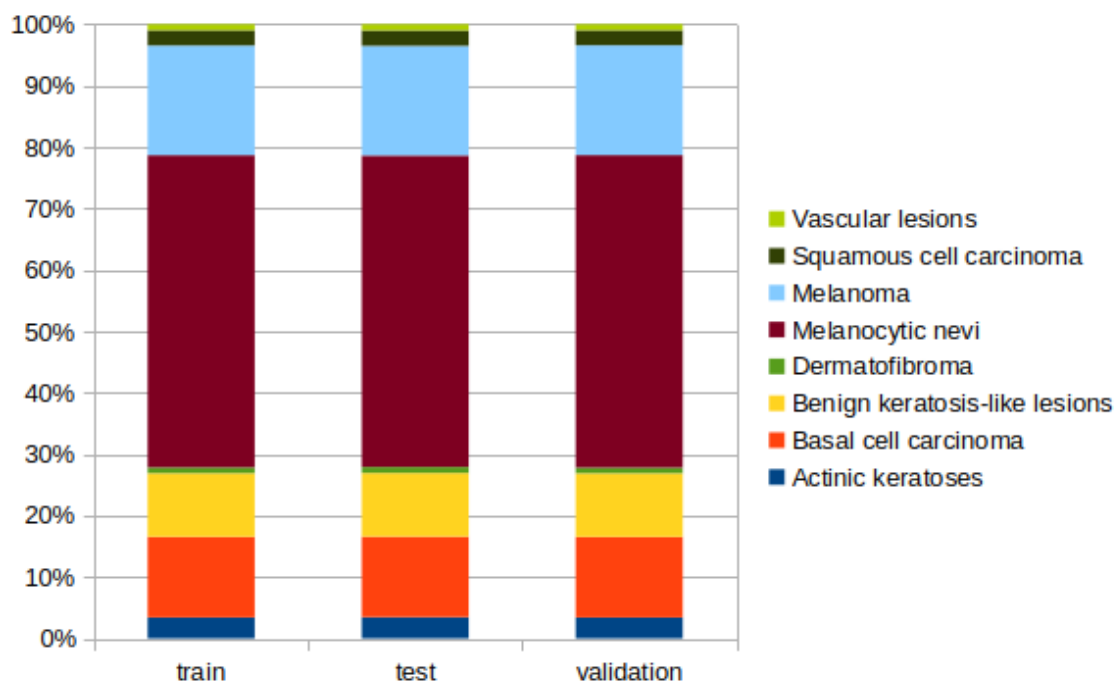
- Osztályok száma: 8
- Képek száma: 25331
- 101 különböző képfelbontás, 576x450-tól, 1024x1024-ig.
- Leggyakoribb felbontások: 1024x1024 (49%), 600x450 (40%), 1024x680 (4%)
- 99 különböző képarány.
- Leggyakoribb képarányok: 1:1 (49%), 4:3 (43%), 128:85 (4%)

Átlag felbontás	Felbontások szórása	Normalizált RGB színcsatornák átgala	Normalizált RGB színcsatornák szórása
854.9 x 760.8	206.9 x 269.7	(0.624, 0.520, 0.504)	(0.242, 0.223, 0.231)

4. táblázat: Módosított SLCD pixel statisztikái



12. ábra: Osztályonkénti összes képek aránya a módosított SLCD adathalmazban.



13. ábra: Osztályok eloszlása felhasználási részhalmazonként belül a módosított SLCD adathalmazban

3.3 Háló tervezés

Azt megelőzően, hogy a dolgozatom végső céljának megfelelő neurális hálót hoznék létre, először próba tanításokat és kiértékeléseket végzek pár próba képosztályozó hálón. Ezek az Irodalomkutatás és technológiák fejezetben ismertetett architektúrákat bemutató cikkekben bemutatott, modellek.

Ezen próba tanítások annak mérésére szolgálnak, hogy a bőrelváltozásokon melyik architektúra teljesít a legjobban. Ennek eredményének fényében fogom a legjobban teljesítő architektúrát arra finomhangolni, hogy az nagy pontossággal tudja egy bőrelváltozásról a melanoma gyanúját megállapítani.

3.3.1 Választott próba hálók

A cikkek mindegyike több prototípusát mutatja be a hálóiknak. A konkrét háló modellek kiválasztásakor azt vettem figyelembe, hogy mindegyik fő architektúra képviselve legyen, a hálók paramétereit között egymáshoz képest ne legyen túl nagy különbség, illetve, hogy a számítási igényük -ennek mértéke a lebegőpontos műveletek száma (floating point operations, FLOP)- akkora legyen, hogy a felhasználói hardverem megbírkozzon vele. Továbbá kijelölök egy architektúrát kontrol példánynak, ami az összehasonlítás alapjául szolgál.

A vázolt elvárásokat a cikkekben bemutatott Base (B), CoAtNet esetén 2-es modellek kielégítik, név szerint: ConvNeXt-B [9], Swin-B [11], CoAtNet-2 [10]. Mindegyik hálóból érhető el 224x224 és 384x384 felbontású képeket befogadó típus, de a véges VRAM-om miatt én egységesen 224² felbontást befogadóakat választottam.

A transzformerek említett adatéhsége és az azt nem kielégítő méretű adathalmazom miatt a transzformer és hibrid hálók esetén mindenképp azok ImageNet-en előtanított változatát vagyok kénytelen használni. Ennek következtében viszont, hogy az összehasonlítás fair maradjon a kontrol példány kivételével végeredményben mindegyik hálóból előtanítottat fogok használni. Az előtanítás mindegyik model esetében a(z) ImageNet-1k adathalmazon történt.

A kontrol példánynak, vagyis az összehasonlítás alapjának pedig szintén ConvNeXt-B-t választok, csak előtanítás nélkül. Ez ugyanis még tisztán a tradicionális CNN architektúra képviselője.

CoAtNet-2 kivételével mindegyik hálózhoz alpból található implementáció a *torchvision models* könyvtárában. CoAtNet-2-höz a *timm* kiegészítő könyvtár biztosít implementációt.

Háló neve	Architektúra	Paraméterszám (millió paraméter)	Számítási igény (GigaFLOP)
ConvNeXt-B	CNN	89	45
Swin-B	Transzformer	88	47.1
CoAtNet-2	CNN+Transzformer	75	49.8

5. Táblázat: Választott, modern architektúrákat képviselő hálók és paramétereik.

3.3.2 Módosítások a hálókon

Mivel mindegyik próbaháló Imagenet adathalmazra (elsősorban Imagenet-1k, de későbbi adathalmazokkal is kísérleteztek) lett tervezve és azon tanítva, alpból 1000 kategóriából osztályoznak, szemben az SLCD-ben található 8 osztállyal.

Célnak és adathalmazbeli különbségnek megfelelően mindegyik modell esetén a kimeneti réteget 8 osztályúra kell cserélni.

3.4 Tanítás

A neurális hálók tanítását konvencionális szkript helyett, a nagyobb rugalmasságért jupyter notebook-ban végzem. Az átláthatóság és hálónként eltérő paraméterek miatt, minden hálót külön notebook-ban.

A notebook-ok egyenként eltérő hálókon dolgoznak, eltérő paraméterekkel és egyszerre tartalmazzák a tanítás és kiértékelés végrehajtását, ezért lehetővé kell tenni, hogy az aktuális futtatásnak megfelelőre legyenek konfigurálhatóak. Kezelnük kell ezen kívül a statisztikák gyűjtését, azok és a hálók mentését külső fájlba és onnan való betöltését.

A különböző notebook-ok között a meggyegyző feladatokat az átláthatóság és könnyebb karbantarthatóság érdekében, felelősségeik szerint csoportosítva több fájlba/modulba tördelem.

3.4.1 Előfeldolgozási lépések

Pytorch *torchvision* könyvtára tartalmaz egy gazdag eszköztárat gépi látáshoz, azon belül a *transforms* modulja egy repertoárt ad kép manipulációhoz. Ennek *Compose* függvénye kényelmes lehetőséget biztosít, a képek modellnek való betöltése előtti előfeldolgozási lépések megadására. Előfeldolgozási pipeline összeállítása a beépített transzformációkon kívül sajátokkal is kiegészíthető.

A próba hálók alapból is négyzetes képeket várnak bemenetül mely képarányt célszerűnek tartok megőrizni a bőrelváltozások alakjai miatt. Így aztán, mivel a módosított SLCD adathalmazban a képarányok változóak, az előfeldolgozás első lépéseként a képarányokat konszolidálok. Mivel a bőrelváltozások lényeges részei a képek közepén találhatóak, ezért négyzetesítés veszteség nélkül megtehető a hosszabik oldal mentén a képszélek levágásával.

A bőrelváltozások orientáció invariánsak. Hogy elősegítsem ennek a megtanulását a tanító (és csak a tanító) halmaz esetén augmentációs lépésként véletlenszerű elforgatást és/vagy vízszintes és/vagy függőleges tükrözés alkalmazását vezetem be. Az elforgatás miatt üresen maradó képsarkak ellensúlyozásáért pedig a forgatás előtt a bemeneti méretnél valamivel nagyobbra skálázom a képet, elforgatás után pedig a középső, bemeneti méretnek megfelelő részt vágom ki.

A validációs és teszt halmaz esetén az előfeldolgozásból kimarad az augmentáció, vagyis a képarány konszolidáció után rögtön a bemeneti méretre skálázható a kép.

Az előfeldolgozás vége pedig, minden halmaz esetén a képet normalizálása és a modellnek bemenetül adható tenzorral alakítása.

3.4.2 Adatbetöltő

Az adathalmazok kiszolgálására Pytorch utils, azon belül data könyvtárában található, Dataset és DataLoader osztály ad lehetőséget. Illetve a torchvision datasets könyvtára tartalmazza pár tipikus adathalmaz leírását. A(z) SLCD is ilyen tipikus, almappákba csoportosított képek halmaza, amelyet kezel a datasets.ImageFolder osztály.

SLCD gyökeréből indulva train/test/val almappákon belül, bőrelváltozásról elnevezett almappákban találhatóak a képek. Ez alapján 3 dataset-et definiálok: train, val, test. Ez ImageFolder osztállyal egyszerűen mindegyikhez a hozzá vezető elérési út megadásával adatható meg:

```
trainSet = datasets.ImageFolder("../SLCD/train", transform=transform)
...
```

A transforms paraméter a képek előfeldolgozásaként végrehajtott műveleteket tartalmazza, ezeket részleteztem a(z) 3.4.1 alfejezetben.

A dataset-eket ezután egy DataLoader-nek kell átadni, ami aztán tanítás során a modellnek átadja a képeket. DataLoader osztály az adathalmazon kívül paraméterként várja még, hogy a képeket megkeverje-e, illetve a batch méret és cpu_worker-ök száma hiperparamétert. Batch méretről 3.4.5 Hiperparaméterek alfejezetben fogok írni. A cpu worker-ök a grafikus kártyára való betöltés előtt előkészítik a képeket, hogy mire betölthetők már előfeldolgozottak legyenek. Ezeket modellenként a batch mérettel megyegyezőre állítom. DataLoader definíció szintaxisára példa:

```
trainLoader = DataLoader(
    trainSet,
    shuffle=True,
    batch_size=batchSize,
    num_workers=workers
)
...
```

DataLoader-ekből is 3-ra van szükség, mindegyik részhalmazhoz 1.

3.4.3 Súlyozott veszteségfüggvény

Több osztályú osztályozóknál a tipikus veszteségfüggvény a Cross-Entropy.

Mivel az adathalmaz kiegyensúlyozatlan, simán ezen való tanítással a modell jó eséllyel hajlamos a többségi osztályok felé húzni. Ez ellensúlyozható a veszteségfüggvényben a ritkább osztályok fel, a gyakoribbak alul súlyozásával.

Minden osztályt a saját, normalizált inverz frekvenciájával súlyozok. Így minden i . osztály w_i súlya:

$$w_i = \text{teljes mintaszám} / (\text{osztályok száma} * \text{osztály mintáinak száma})$$

Cross-Entropy súlyozott esetben:

$$CE = -\sum_i w_i y_i \log(y_i')$$

$i=1$ -től osztályok számáig,

w_i az i . osztály súlya, y_i a valódi bement, y_i' a modell kimenete.

Pytorchban több veszteségfüggvény, beleértve a súlyozható Cross-Entropy-t is előre implementálva van.

3.4.4 Optimalizáló

Tanítás során a háló tévedéseinek kiszámítása után az optimalizáló frissíti a háló súlyait. Modern CNN-nél, transzformereknél bevett az AdamW használata, a próba hálókat bemutató cikkek mindegyikében is ezt használták. AdamW egy fontos módszere a tanítás javítására, hogy weight decay-t lehet neki megadni, ami a nagyra növvő aktivációs súlyokat hivatott visszafogni, hogy a modell ne tanuljon túl.

3.4.5 Hiperparaméterek

Sajnos a hiperparaméterek meghatározására nincsenek univerzális képletek, ráadásul sok mindentől függ melyik pontos érték a legmegfelelőbb. Legfeljebb az optimizáló fajtájától és modell architektúra alapján ökölszabályszerűen bevált kiinduló értékeket lehet találni.

Legbiztosabb alapnak azt találtam, ha mindegyik hálózhoz az öt bemutató szaklapok hiperparamétereit veszem alapul. Weight-decay esetén ez módosítás nélkül megtehető, batch méret esetén viszont a GPU-m memóriája limitál, így ennek csökkenésével lineárisan le kell skálázom a learning-rate-t is.

Fontos kiemelni, hogy a szaklapok a pretraining és fine-tuning folyamatokra eltérő hiperparamétereket használtak. Az én esetemben ez előtanított modelleket veszem át, így a céloknak való áttanítás már inkább felel meg fine-tuning-nak, így az ahhoz tartozó hiperparamétereket vettem alapul. Kiegészítve az összehasonlítási alap, előtanítatlan ConvNeXt számára a rendes tanításhoz tartozóakkal.

Háló neve	ConvNeXt-B (előtanítatlan)	ConvNeXt-B	Swin- Transformer-B	CoAtNet-2
Batch méret	4096	512	1024	512
Learning-rate	$4 \cdot 10^{-3}$	$5 \cdot 10^{-5}$	10^{-3}	$5 \cdot 10^{-5}$
Weight-decay	$5 \cdot 10^{-2}$	10^{-8}	$5 \cdot 10^{-2}$	10^{-8}

6. táblázat: Szaklapokban használt hiperparaméterek összefoglalása, hálónként. ConvNeXt esetén az előtanítása (előtanítatlan oszlop) és a fine tuningra használtakkal együtt.

Learning-rate-k lineáris skálázáshoz a szaklapban szereplő, és az én GPU-m által elbírt batch méret hányadosát használtam. Ezek pontosan: ConvNeXt futtatása esetén 16, Swin és CoAtNet esetén: 8. A skálázott eredményt végül felfelé kerekítettem.

Háló neve	ConvNeXt-B (előtanítatlan)	ConvNeXt-B	Swin- Transformer-B	CoAtNet-2
Batch méret	16	16	8	8
Learning-rate	$2 \cdot 10^{-5}$	$2 \cdot 10^{-6}$	$8 \cdot 10^{-6}$	$8 \cdot 10^{-7}$
Weight-decay	$5 \cdot 10^{-2}$	10^{-8}	$5 \cdot 10^{-2}$	10^{-8}

7. táblázat: Általam használt hiperparaméterek összefoglalása, hálónként.

Hiperparaméterek között szokták még említeni a tanulási ciklosok számát. Ezt én nem rögzítem előre, helyette a validációs veszteség csökkenésének megállásakor állítom le a tanítást.

Szintén hiperparaméterek témakörében azt is fontos megemlíteni, hogy ezek az értékek nem kőbe vésettek, a tanítás során finomhangolhatóak különböző stratégiák szerint. Ilyeneket például a hivatkozott szaklapokban is alkalmaznak. Én azonban, mivel első körben csak azt döntöttem el, melyik architektúra szolgál majd a melanoma detektor alapjául, nem változtatom a hiperparamétereket a kiindulási értékükhöz képest.

3.4.6 Értékelési metrikák

Már említettem, hogy a százalékos pontosság akkor is lehet magas, ha a kis mintaszámú osztályokat csak ignorálja a modell. Ez az adathalmaz kiegyensúlyozottságnak fontosságán kívül arra is felhívja a figyelmet, hogy a találati pontosság százalékos aránya (accuracy) sok esetben nem elég pontos mérése egy modell teljesítményének. Sőt! Számos árulkodóbb metrika létezik neurális hálókhoz. Módszerükön kívül még egy fontos különbség köztük, hogy milyen feladatra (osztályozás, objektum detektálás, kép szegmentáció) tervezett hálók esetén relevánsak.

Az én dolgozatomban esetében osztályozókat vizsgálok, azon belül is elsősorban több (osztályok száma ≥ 3) osztályúakat, de érdekesek lehetnek még a 2 osztályú (bináris), pozitív/negatív választ adó osztályozók esetén használt metrikák is.

Osztályozók esetén, metrikákhoz a kiindulás a confusion matrix, amiben táblázatosan egymás mellé állítjuk, hogy a melyik tényleges osztályhoz melyiket hányszor jósolta a modell. Vagyis minden n_{ij} elem annak a száma, hányszor jósolta a modell a j -ik osztályt, miközben a bemenet az i -ik osztály volt. Ennek átlójából a helyes találatok olvashatóak ki, egy

adott tényleges osztály oszlopa alatti sorokból pedig, hogy az adott osztályú bemenetet mely másikkal és hányszor keverte össze.

Confusion mátrix-ból további százalékos metrikák is számíthatóak az egyes osztályokat érintő pontosságra, ezekhez azonban a tévedéseket össze kell vonni. Ennek intuitív módja, hogy a mért i -ik osztályhoz egy -bináris esettel megegyező- 2×2 -es, pozitív/negatív mátrixot állítunk elő, aminek cellái $a(z)$: true positive (TP), true negative (TN), false positive (FP), false negative (FN) értékeket reprezentálják. A teljes confusion mátrix n_{ij} értékei alapján, ezen összegző mátrix mezőinek előállítására szolgáló képleteket foglaltam össze a következő táblázatban, egy elképzelt i -ik osztályra.

Tényleges →	Pozitív	Negatív
↓ Jósolt		
Pozitív	$TP = n_{ij}$	$FN = \sum_{j \neq i} n_{ij}$
Negatív	$FP = \sum_{j \neq i} n_{ji}$	$TN = \sum_{j \neq i} \sum_{h \neq i} n_{jh}$

8. táblázat: Egy modell i -ik osztályra adott tényleges/téves pozitív/negatív jóslatait összegző képletek [14]

Az [8.] táblázat képleteit a következő képpen foglalnám össze közérthetőbben: Legyen az osztályunk 'X', ekkor: TP = hányszor válaszolt helyesen X bementre X-t a modell; FN = hányszor válaszolt X bementre X-től eltérőt; FP = hányszor válaszolt X-től eltérő bementre X-et; TN = hányszor válaszolt X-től eltérő bemenetre, X-től eltérőt;

Ennek a 2×2 -es mátrixnak az ismeretében pedig a modell 1 osztályt érintő pontosságát mérő metrikák a következők:

- Accuracy: Nem összes, hanem 1 specifikus osztályra vett pontosság. Annak aránya, hogy X bemenet esetén hányszor találta el, X-től eltérő esetén pedig hányszor mondott szintén X-től eltérőt az összes predikcióhoz képest.

$$\text{accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

- Recall: Azt méri, egy specifikus osztály elemeiből mennyit talált meg a modell.

$$\text{recall} = TP / (TP + FN)$$

- Precision: Azt méri, azokból az elemekből, amit a modell valamilyen osztályúnak mondott, abból mennyi volt ténylegesen az az osztály.

$$\text{precision} = TP / (TP + FP)$$

- F1-score: Recall és precision kombinációja.

$$F1 = (2 * precision * recall) / (precision + recall)$$

[14]

3.4.7 Mentett állapot és statisztikák

A tanítás megvalósító programrészeknek tanulási ciklusonként el kell menteniük a modellek aktuális állapotát, ezzel garantálva, hogy bármilyen megszakadás esetén onnan lehessen a tanítást folytatni, ahol az abba maradt. Az utolsó állapot mellett, pedig a validáció eredménye alapján egy legjobban teljesítő háló mentést is készíteni kell, hogy túltanulás esetén is megmaradjon az általánosítani jobban tudó háló.

A háló súlyokon kívül a visszatölthetőségért még el kell menteni az eddig végrehajtott tanítási ciklusok számát, illetve az optimalizáló állapotát.

Statisztikai célokra pedig minden tanítási ciklusról rögzítem a(z): adott ciklus számát, tanítási halmazon a veszteség és pontosság értékét, validációs halmazon a veszteség és pontosság értékét; illetve hogy még részletesebb képet kapjunk az adott állapot teljesítményéről: a validációs halmaz confusion mátrixát.

3.4.8 Megosztható erőforrások

A konkrét megvalósítást és a tanítás notebookban való futtatását szignifikánsan leegyszerűsíti, ha a notebookok között ismétlődő feladatokat külső, összes notebook által felhasználható modulokba szervezem ki.

Főbb ismétlődő feladatok:

- Modell tanítása és annak részfeladatai
- Modell kiértékelése
- Eredmények, grafikonok megjelenítése
- Statisztikák rögzítése
- Erőforrások mentése, betöltése

Ezen feladatokat indokolt kiszervezni olyan módon, hogy a konkrét modell/notebook specifikus erőforrások számukra parametrikusan átadhatóak legyenek.

Notebook/modell specifikus feladatok:

- Model és paramétereinek inicializálása
- Folyamatok végrehajtásának konfigurálása

Ezek a feladatok pedig bár notebookonként és futtatásonként változnak, azért, hogy a megosztott feladatokra gyakorolt hatásuk átlátható és kezelhető legyen, valójában ezeknek is szükséges egységes leírást adni.

4 Implementáció

Megválasztott erőforrások és paraméterek, megvalósítandó feladatokkal szembeni követelmények meghatározását követően, ebben a fejezetben részletezem az elkészített azok megvalósítását.

4.1 Training package és moduljai

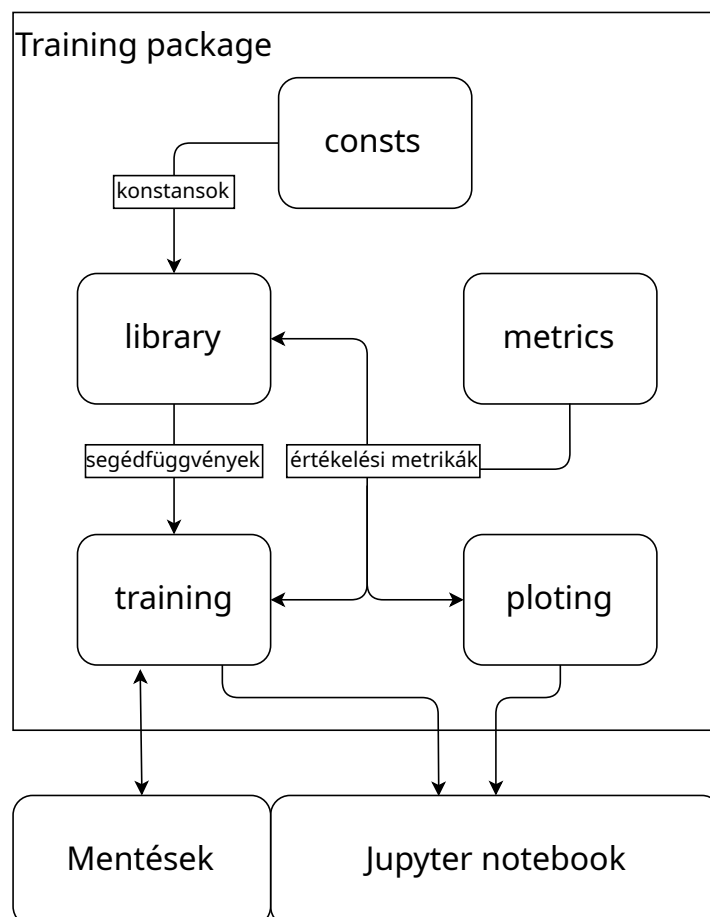
A bármely tanítás során felhasználható segédfüggvényeket, változókat, osztályokat, a feladataik szerint az alábbi modul fájlokba csoportosítottam, amiket együttesen egy **Training** elnevezésű mappába rendeztem, ami így egy python package-ként használható:

- **consts.py:** Modellek között megegyező, tanítás során nem változó konstans értékek gyűjteménye. Itt van rögzítve az adathalmaz elérési útja, a képek normalizáláshoz kiszámított pixel statisztikák és a bemeneti képméret.
- **metrics.py:** Confusion mátrix műveleteket (létrehozás, kilapítás, visszafejtés) és a modellek értékelésére használt metrikák képleteit tartalmazza.
- **library.py:** A tanítás és modell inicializáláshoz releváns segédfüggvények gyűjteménye. Itt található a(z): előfeldolgozásokat komponáló, adathalmazból modellbe adatbetöltőket definiáló, illetve a modellen 1 db tanítási vagy validációs ciklust végrehajtó segédfüggvények.
- **training.py:** A tanítást futtató notebookok számára tartalmaz, a tanítás folyamat egészét átfogó, azok erőforrásait kezelő osztályokat és a tanítást leíró függvényt. Ezen átfogó osztályok:
 - **Config adatosztály:** A notebook illetve modell specifikus, a futtatás folyamat szabályozó beállításokat formalizálja. A tanítás hiperparamétereit is ez az osztály tárolja. A mentések rendszerezéséhez tárolja még a modell nevét.
 - **TrainingResources osztály:** A modell tanítás erőforrásait foglalja össze, azokhoz a notebookok/modellek között megegyezőket rögtön magában inicializálja. Tartalmaz metódust a mentésre és betöltésre.
- **plotting.py:** Segédfüggvényeket tartalmaz grafikonok és metrikák megjelenítésére és azok fájlba mentésére.

4.1.1 Training package struktúrája

Az általánosabb (al)modulokat már a package-en belül felhasználják az átfogóbb modulok. Vagyis, lényegében, az átfogóbb modulok építkeznek az almodulokból, hogy a package-ből kifelé egy leegyszerűsített API-t tudjanak biztosítani az őket futtató notebook számára.

- Consts-ban definiált konstansokat a library modul használja
- Metrics modul confusion mátrix kezelő függvényeit és a metrikák képleteit a library, training és plotting modulok is használják
- Library segédfüggvényei a training modulban kerülnek felhasználásra
- Training modul kommunikál a fájlrendszerrel és ad interfészt a jupyter notebooknak.
- Plotting modul eszköztárat biztosít az eredmények notebookban való megjelenítésére



14. ábra: Training package belső struktúrája, modulok által biztosított funkcionálisok összefoglalása.

A package fő struktúráját és a belső, egyszerűbb modulok szolgáltatásait az átfogóbbak felé, majd azokat a package-n kívülre szemlélteti a 14. ábra, de fontos megjegyezni, hogy a modulok mindegyike és az általuk biztosított segédfüggvények önállóan is használhatóak. Erre szemléletes use-case, ha a fő tanító ciklus után a teszt halmazon már csak 1 önálló validációs ciklust kell végrehajtani, ahhoz a *library* modulban található a *validate* függvény.

4.2 Futtatás jupyter notebook-ban

A Config osztály beállításai foglalják keretbe a egy notebookon belül a futás szabályozását. TrainingResources tartalmazza a tanított modellt és az ahhoz tartozó további erőforrásokat, aktuális állapotuk és statisztikáik. Ezek segítségével, minden jupyter notebookban boilerplate szerűen írható le a tanítás és értékelés. Ennek lépései példákkal kiegészítve:

1. Config inicializálása a modell nevével. Beállítások és hiperparaméterek megadása.

```
c = Config("Swin-B")
c.purge = False #Korábbi tanítás törlése
c.training = True
c.loadBest = False #Utoljára tanított/legjobb betöltése

#Hiperparaméterek
c.learningRate = 8e-8
c.weightDecay = 1e-8
c.batchSize = 8
c.workers = 8
```

2. Használni kívánt modell architektúra inicializálása Pytorch és/vagy timm segítségével.

```
from torchvision.models import swin_b, Swin_B_Weights
model = swin_b(weights=Swin_B_Weights)
model.head = torch.nn.Linear(model.head.in_features, c.numClasses)
```

3. TrainingResources inicializálása paraméterként a modellel, korábbi állapot betöltése.

```
TR = TrainingResources(c, model, initClassifierTrainers, device)
TR.load(c)
```

4. Tanító függvény futtatása.

```
trainingLoop(c, TR)
```

5. A tanítás végeztével a modell kiértékelhető. Ehhez már bele kell nyúlni a library modul *validate* segédfüggvényéhez. A legjobban teljesítő modell tesztelése a *loadBest* átállításával, és újbóli betöltéssel tehető meg.

```
c.loadBest = True
```

```

TR.load(c)
test_loss, test_acc, CM = validate(
    TR.model, TR.test_loader, TR.criterion, device
)

```

4.2.1 Modellek inicializálása

A próba architektúrák mindegyikének inicializálásra eszközt biztosítanak a használt könyvtárak, és még azok módosítását is könnyen teszik lehetővé. A hálók inicializálása, és az osztályozó kiemelteinek száma a következően néz ki:

- **ConvNeXt-B (előtanított, előtanítatlanhoz a *weights* paraméter kihagyandó):**

```

from torchvision.models import convnext_base, ConvNeXt_Base_Weights

model = convnext_base(weights=ConvNeXt_Base_Weights)
model.classifier[2] = torch.nn.Linear(
    model.classifier[2].in_features, out_features=c.numClasses
)

```

- **Swin-B (előtanított):**

```

from torchvision.models import swin_b, Swin_B_Weights

model = swin_b(weights=Swin_B_Weights)
model.head = torch.nn.Linear(model.head.in_features, c.numClasses)

```

- **CoAtNet-2 (előtanított):**

```

from timm import create_model

model = create_model(
    "coatnet_2_rw_224",
    pretrained=True,
    num_classes=c.numClasses
)

```

5 Próba hálók eredményei

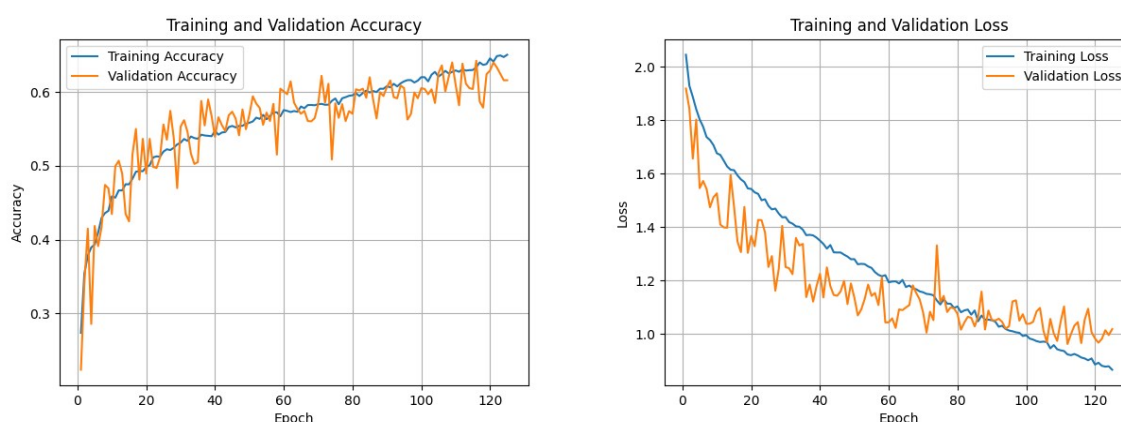
Az 4.2 alfejezetben ecsetelt mintát követve létrehoztam 1-1 tanítást és kiértékelést megvalósító notebookot mindegyik próbahálónak. Az eredményeket és háló súlyokat egy *Models/* kimeneti mappába mentettem, mindegy modell esetén egy almappába, beleértve a folyamatokat és eredményeket szemléltető grafikonokat is. Ebben a fejezetben ezeket az eredményeket vizsgálom.

5.1 Tanítások eredményei

Minden háló esetén röviden megvizsgálom a tanulás folyamatát.

5.1.1 Előtanítatlan ConvNeXt-B, összehasonlítási alap

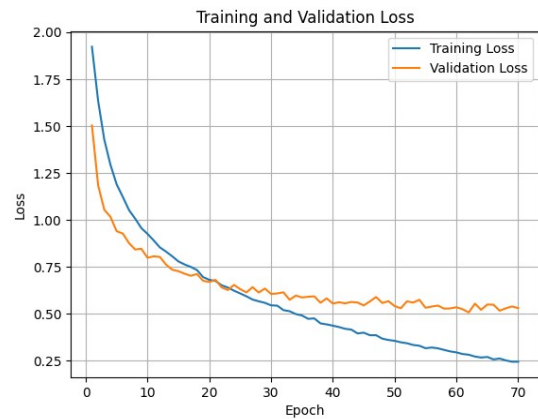
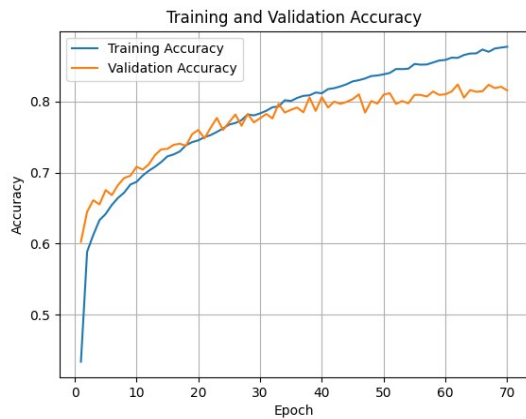
125 ciklusig hagytam tanulni, amikortól a validáció már nem javult tovább. A legerősebb validációs értéket 116-ik ciklusban érte el.



15. ábra: Pontosság és veszteség változása a tanító és validációs képhalmazon, előtanítatlan ConvNext-B tanítási ciklusainak előrehaladtával.

5.1.2 Előtanított ConvNeXt-B

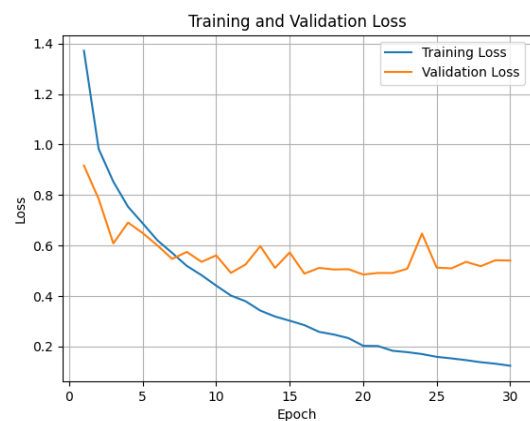
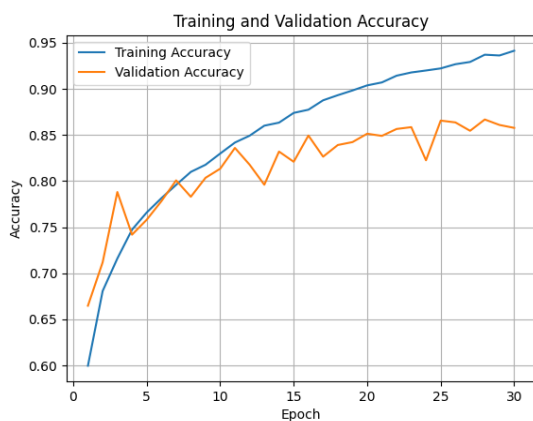
70 ciklusig hagytam tanulni. 20-ik ciklus körülől a validáció lelassult a javulásban, de nem hagyta abba, a legjobb eredményt a 62-ik ciklusban érte el.



16. ábra: Pontosság és veszteség változása a tanító és validációs képhalmazon, előtanított ConvNext-B tanítási ciklusainak előrehaladtával.

5.1.3 Előtanított SwinTransformer-B

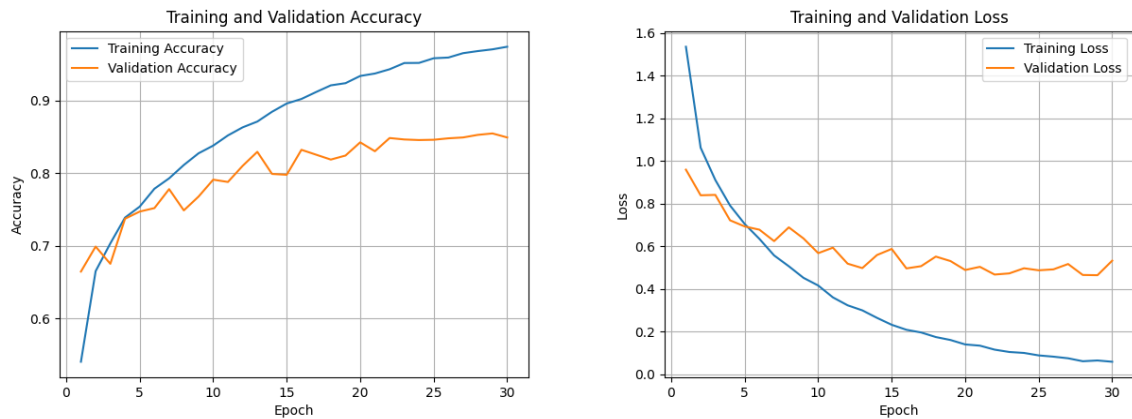
Nagyon gyorsan konvergált, csak 30 ciklusig hagytam tanulni. A validációs veszteség már a 10. ciklustól lelassult a növekedésben, de a pontosság még növekedett, ezért nem állítottam le hamarabb. A legerősebb pontosságot a 28. ciklusban érte el.



17. ábra: Pontosság és veszteség változása a tanító és validációs képhalmazon, előtanított Swin-B tanítási ciklusainak előrehaladtával.

5.1.4 Előtanított CoAtNet-2

Szintén gyorsan konvergált, így ezt is csak 30 ciklusig hagytam tanulni. A 20 ciklustól a validációs veszteség és pontosság is szinte kilapult, nagy ritkán rúgott egy kicsit följebb. A legnagyobb pontosságot a 29. ciklusban érte el.



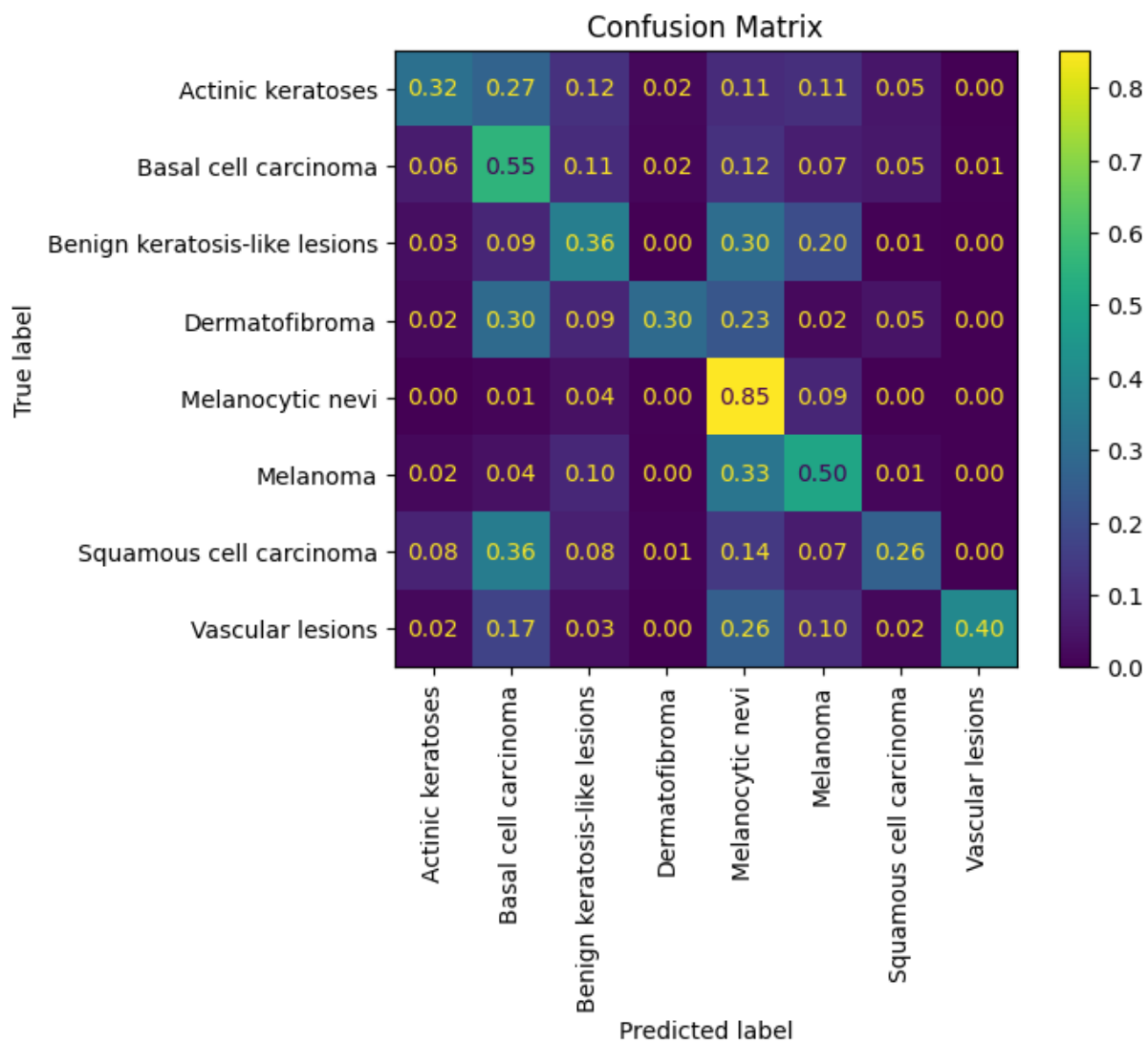
18. ábra: Pontosság és veszteség változása a tanító és validációs képhalmazon, előtanított CoAtNet-2 tanítási ciklusainak előrehaladtával.

5.2 Teszt eredmények

Minden háló esetén részletesen megvizsgálom azok teljesítését a bőrelváltozások megkülönböztetésében. Ez az elsődleges szempont, mely alapján finomhangolásra választom a győztes hálót, ezen belül is, a legfőbb szempont, hogy melanoma-t mennyire tudta jól kiszűrni még hozzá különösen hasonló bőrelváltozások (pl.: anyajegy) esetén is.

A teljesítmények vizsgálatára minden hálót annak legjobb validációt elérő súlyaival töltök be, majd ezen a hálón áteresztmem a teszt halmaz képeit. Az eredményeket confusion mátrixba gyűjtöm, a megjelenítéshez azok értékeit normalizáltam. A confusion mátrix alapján pedig egy segédfüggvényem minden osztály esetében kiszámolja arra a(z): accuracy, precision, recall, F1 score metrikákat. Ezeket százalékos formában táblázatokba foglaltam össze.

5.2.1 Előtanítatlan ConvNeXt-B, összehasonlítási alap teljesítménye

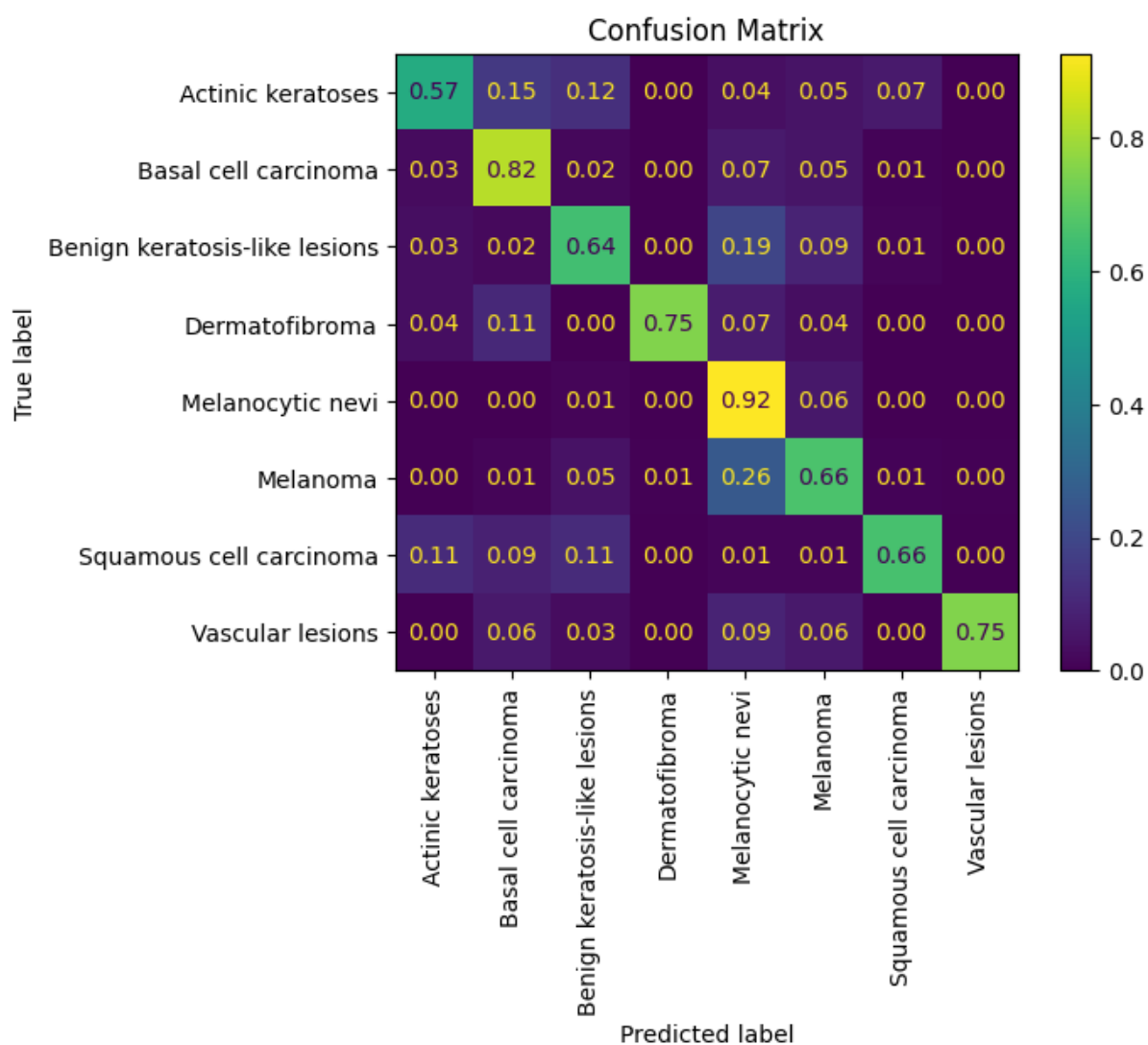


19. ábra: Előtanítatlan ConvNeXt-B confusion mátrixa.

	accuracy	precision	recall	f1
Actinic keratoses	94.88 %	31.58 %	40.91 %	35.64 %
Basal cell carcinoma	88.31 %	55.49 %	54.65 %	55.07 %
Benign keratosis-like lesions	86.34 %	36.27 %	42.21 %	39.02 %
Dermatofibroma	98.31 %	29.55 %	52.0 %	37.68 %
Melanocytic nevi	80.08 %	85.16 %	73.52 %	78.92 %
Melanoma	82.24 %	50.21 %	52.32 %	51.24 %
Squamous cell carcinoma	95.51 %	26.42 %	43.75 %	32.94 %
Vascular lesions	98.5 %	39.66 %	88.46 %	54.76 %

9. táblázat: Előtanítatlan ConvNeXt-B, teszt halmazon elért metrikái, osztályonként.

5.2.2 Előtanított ConvNeXt-B teljesítménye



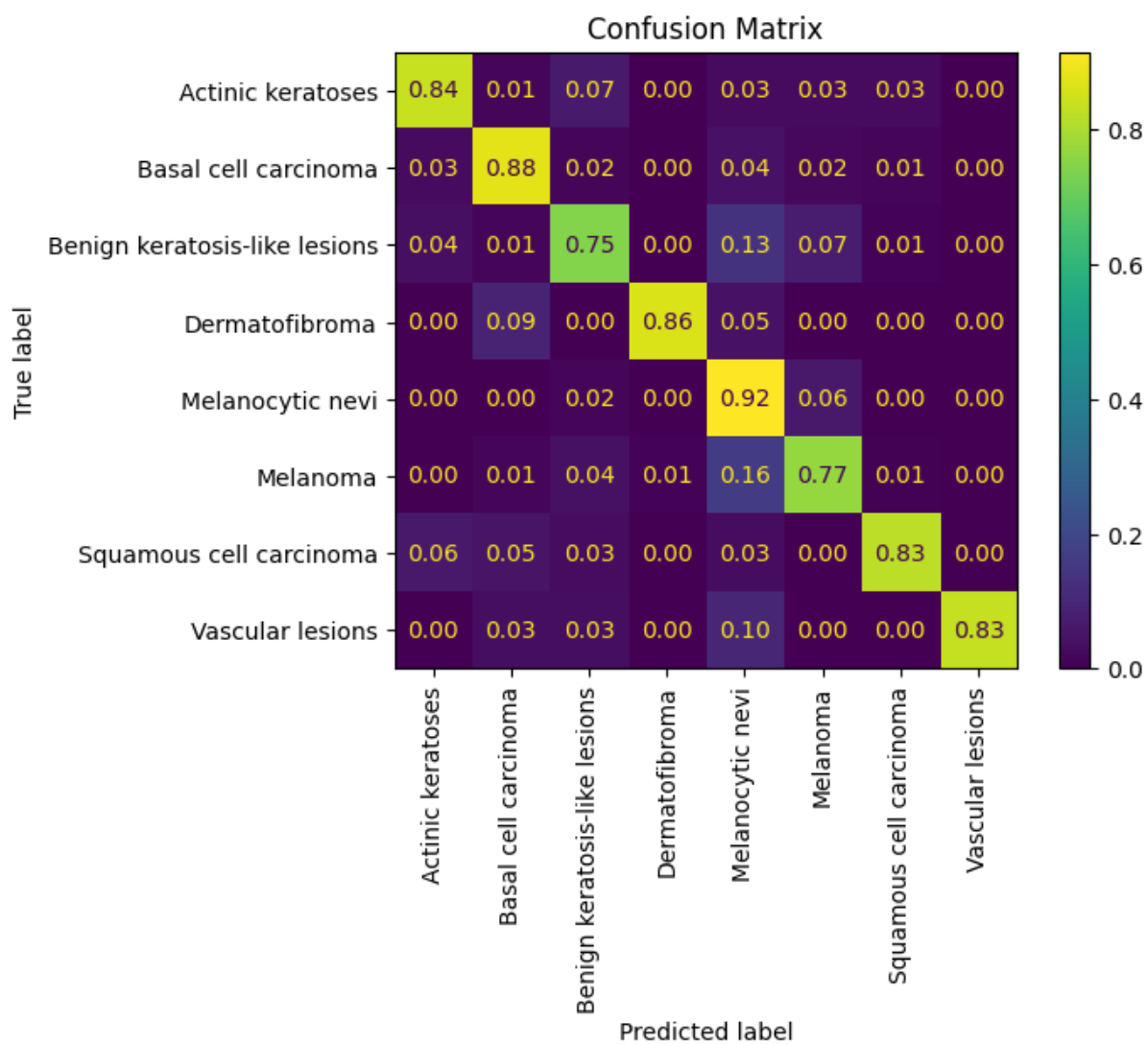
20. ábra: Előtanított ConvNeXt-B confusion mátrixa.

	accuracy	precision	recall	f1
Actinic keratoses	97.09 %	56.73 %	67.05 %	61.46 %
Basal cell carcinoma	95.87 %	82.39 %	87.09 %	84.67 %
Benign keratosis-like lesions	93.07 %	64.36 %	74.14 %	68.9 %
Dermatofibroma	99.57 %	75.0 %	84.0 %	79.25 %
Melanocytic nevi	87.83 %	92.45 %	82.76 %	87.34 %

Melanoma	88.54 %	66.27 %	72.85 %	69.4 %
Squamous cell carcinoma	98.35 %	65.71 %	71.88 %	68.66 %
Vascular lesions	99.61 %	75.0 %	92.31 %	82.76 %

10. táblázat: Előtanított ConvNeXt-B, teszt halmazon elért metrikái, osztályonként.

5.2.3 Előtanított SwinTransformer-B teljesítménye



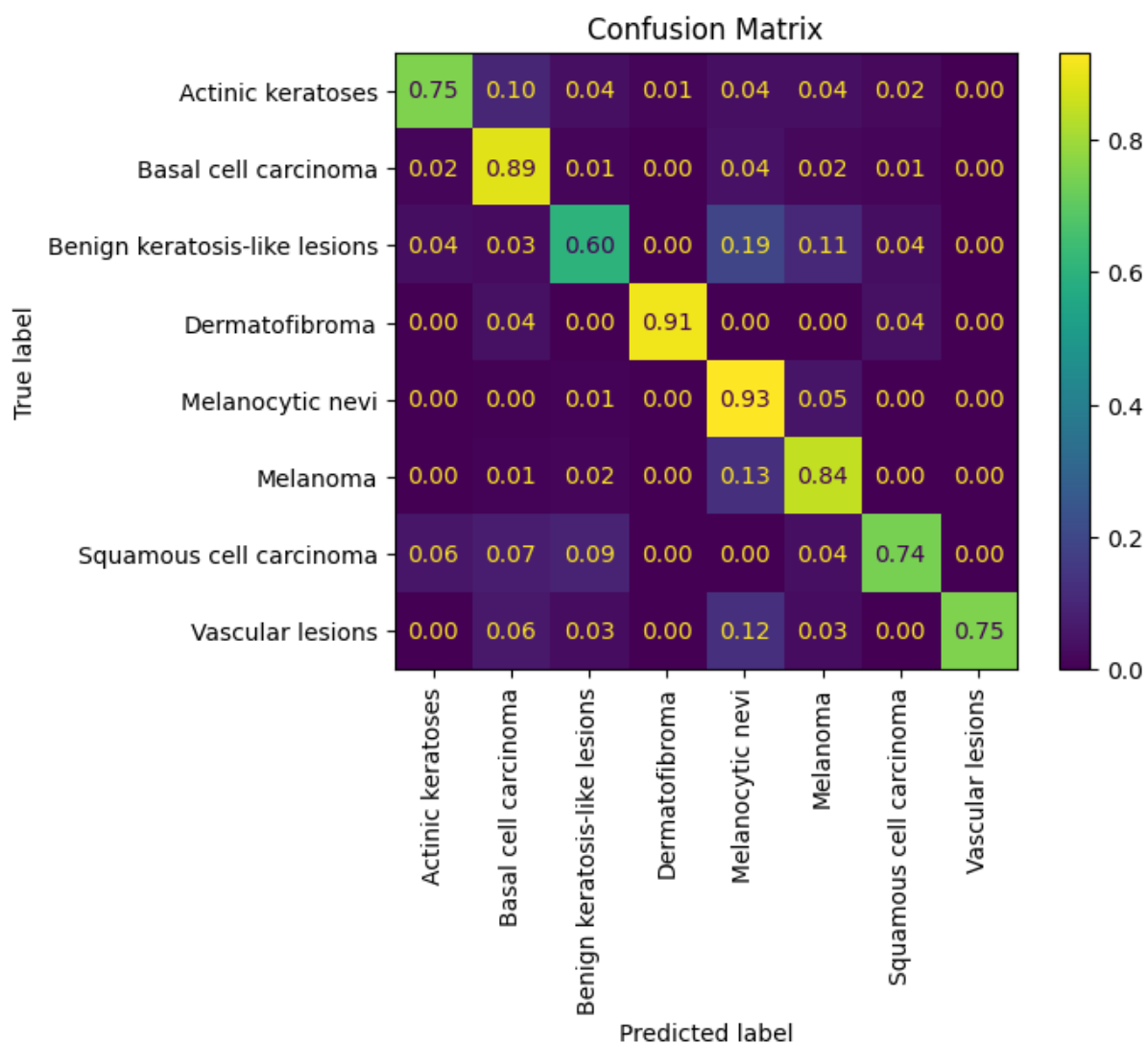
21. ábra: Előtanított Swin-B confusion mátrixa.

	accuracy	precision	recall	f1
Actinic keratoses	98.54 %	84.0 %	71.59 %	77.3 %
Basal cell carcinoma	97.52 %	87.71 %	94.29 %	90.88 %
Benign keratosis-like lesions	95.04 %	74.55 %	79.09 %	76.75 %
Dermatofibroma	99.65 %	86.36 %	76.0 %	80.85 %
Melanocytic nevi	90.59 %	91.53 %	89.75 %	90.63 %

Melanoma	91.69 %	76.89 %	76.38 %	76.63 %
Squamous cell carcinoma	99.09 %	82.54 %	81.25 %	81.89 %
Vascular lesions	99.76 %	83.33 %	96.15 %	89.29 %

11. táblázat: Előtanított Swin-B, teszt halmazon elért metrikái, osztályonként.

5.2.4 Előtanított CoAtNet-2 teljesítménye



22. ábra: Előtanított Swin-B confusion mátrixa.

	accuracy	precision	recall	f1
Actinic keratoses	98.35 %	78.05 %	72.73 %	75.29 %
Basal cell carcinoma	97.17 %	85.95 %	93.69 %	89.66 %
Benign keratosis-like lesions	95.24 %	76.3 %	78.33 %	77.3 %
Dermatofibroma	99.69 %	90.48 %	76.0 %	82.61 %
Melanocytic nevi	90.79 %	91.56 %	90.14 %	90.85 %
Melanoma	91.65 %	76.96 %	75.94 %	76.44 %
Squamous cell carcinoma	99.02 %	84.21 %	75.0 %	79.34 %
Vascular lesions	99.69 %	78.12 %	96.15 %	86.21 %

12. táblázat: Előtanított CoAtNet-2, teszt halmazon elért metrikái, osztályonként.

5.3 Legjobban teljesítő architektúra

6 Melanoma detektor

A következő fejezetben tervezett modell a célja már egy bőrelváltozásról kizárólag annak megállapítása, az melanoma-e vagy sem. Ennek megfelelően tervezem át és finomhangolom a próbákön legjobban teljesítő, ___ architektúrát, arra optimalizálva, hogy az adathalmazban a lehető legtöbb melanoma-t megtalálja, vagyis metrikák közül a melanoma recall-ját szándékozom maximalizálni.

6.1 architektúra_neve áttekintése

A már eleve előtanított ___ architektúra belső rétegeit a próba tanítások is tovább finomították, egy bőrelváltozásnál milyen mintázatokra kell figyelni. A megtanult mintázatok kár lenne veszni hagyni, így nem mindegy, hogyan hajtom végre a következő finomhangolást.

Ahhoz, hogy a belső rétegekben megtanult mintázatok ne vesszenek el, csak transzfer learning-el egy új feladatra alkalmazza azokat, a modell rétegein befagyasztom a tanulást, kizárólag a kimeneti osztályozót cserélem le.

A következő alfejezetekben ezt és az ebből következően szükségessé váló további változtatásokat részletezem.

6.1.1 Adatbetöltő módosítása

Az új esetben pusztán a szerint kell címkézni az adatokat, hogy az melanoma-e vagy sem. Ez megtehető az adathalmaz fájlrendszeren való módosítása nélkül is, pusztán a Pytorch dataset-ek és dataloader módosításával.

Bináris adathalmazhoz *BinaryMelanomaDataset* osztályt definiálok, ami a Pytorch Dataset osztályából öröklődik. Az *__getitem__* elem-lekérdező függvényének felülírásával át lehet címkézni az osztályokat, ugyanis a DataLoader is ezzel a függvénnyel kérdezi le a képeket mielőtt azt továbbadja a modellnek.

A felülírt függvényben a melanoma osztálynevű képekhez az 1 címkét, minden más osztályhoz a 0 címkét rendelem.

Ennek megvalósítása:

```
class BinaryMelanomaDataset(Dataset):
```



```

...
def __getitem__(self, index):
    img, originalLabel = self.dataset[index]
    newLabel = 1 if originalLabel == self.melanoma_idx else 0
    return img, newLabel
...

```

6.1.2 Osztályozó

A több osztályú kimenet helyett áttálok egy bináris, melanoma igen/nem kimenetre.

Laikusan belátható, hogy melanoma diagnosztizálás esetén, egy hamis negatív eredmény végzetesebb lehet, mint egy hamis pozitív. Ezen kockázathoz való hozzáállást szeretném tükrözni a saját melanoma detektoromban is, azáltal, hogy az osztályozó kimenetül kisebb gyanú esetén is inkább melanoma-t jósoljon.

6.1.3 Veszteség és optimalizáló

Optimalizáló esetén a(z) AdamW bináris esetben is megállja a helyét.

Veszteségfüggvény esetén viszont bár működőképes a sima CrossEntropy, mint „2 osztályú” osztályozón, a standard a „Binary Cross-Entropy with logits”, Pytorch-ban: BCEWithLogitsLoss.

Képlete:

6.2 Áttanítás

Az osztályozó és így a kimenet változása ellenére, a több osztály által megtanult mintázatok továbbra is hasznosak,

6.3 Eredmények

7 Irodalomjegyzék

- [1] Maria Trigka, Elias Dritsas, „A Comprehensive Survey of Machine Learning Techniques and Models for Object Detection,” január 2025. [Online]. Available: <https://www.mdpi.com/1424-8220/25/1/214> [Hozzáférés dátuma: 27 március 2026].
- [2] Mingle Xu, Sook Yoon, Alvaro Fuentes, Dong Sun Park, „A Comprehensive Survey of Image Augmentation Techniques for Deep Learning” november 2022. [Online]. Available: <https://arxiv.org/pdf/2205.01491>. [Hozzáférés dátuma: 27 március 2026].
- [3] IBM, „The 2026 Guide to Machine Learning ” 2026. [Online]. Available: <https://www.ibm.com/think/machine-learning#605511093>. [Hozzáférés dátuma: 05 május 2026].
- [4] Wikipedia, „Kernel (image processing)” [Online]. Available: [https://en.wikipedia.org/wiki/Kernel_\(image_processing\)](https://en.wikipedia.org/wiki/Kernel_(image_processing)). [Hozzáférés dátuma: 06 május 2026].
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, Illia Polosukhin, „Attention Is All You Need” 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf. [Hozzáférés dátuma: 06 május 2026].
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby, „AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE” 2021. [Online]. Available: <https://arxiv.org/pdf/2010.11929/100>. [Hozzáférés dátuma: 07 május 2026].
- [7] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, „Deep Residual Learning for Image Recognition” december 2015. [Online]. Available: <https://arxiv.org/pdf/1512.03385v1>. [Hozzáférés dátuma: 10 április 2026].

- [8] Dr. Szegletes Luca, „Geometriai és topologikus mélytanulás” 2026. tavasz.
- [9] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie, „A ConvNet for the 2020s” Március 2022. [Online]. Available: <https://arxiv.org/pdf/2201.03545>. [Hozzáférés dátuma: 14 Április 2026].
- [10] Zihang Dai, Hanxiao Liu, Quoc V. Le, Mingxing Tan, „CoAtNet: Marrying Convolution and Attention for All Data Size” szeptember 2021. [Online]. Available: <https://arxiv.org/pdf/2106.04803>. [Hozzáférés dátuma: 14 Április 2026].
- [11] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, Baining Guo, „Swin Transformer: Hierarchical Vision Transformer using Shifted Windows” augusztus 2021. [Online]. Available: <https://arxiv.org/pdf/2103.14030>. [Hozzáférés dátuma: 14 április 2026].
- [12] Pablo Lopez Santori, „Melanoma Detection Dataset” 2020. [Online]. Available: <https://www.kaggle.com/datasets/wanderdust/skin-lesion-analysis-toward-melanoma-detection>. [Hozzáférés dátuma: 25 Április 2026].
- [13] Ahmed AL-Rufai, „Skin Lesions Classification Dataset” 2024. [Online]. Available: <https://www.kaggle.com/datasets/ahmedxc4/skin-ds>. [Hozzáférés dátuma: 25 Április 2026].
- [14] Oona Rainio, Jarmo Teuho & Riku Klén, „Evaluation metrics and statistical tests for machine learning” március 2024. [Online]. Available: <https://www.nature.com/articles/s41598-024-56706-x>. [Hozzáférés dátuma: 26 Április 2026].
- [x] Szerző, „Cím” június 2009. [Online]. Available: [url](#). [Hozzáférés dátuma: 20 Március 2026].

Függelék

7.1 Függelék

Nyilatkozat generatív mesterséges intelligencia alkalmazásáról¹

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☐ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás			
Prompt lényegi része			
Programkód generálása			
Prompt lényegi része			
Új ötletek, megoldási javaslatok generálása			
Prompt lényegi része			
Vázlat létrehozása (szövegstruktúra, vázlatpontok)			
Prompt lényegi része			
Szövegblokkok létrehozása	-	-	-
Prompt lényegi része	-		
Képek generálása illusztrációs célból	-	-	-

¹ <https://vik.bme.hu/hallgatoknak/altalanos/mi-hasznalat-ajanlasok>

Prompt lényegi része	-		
Adatvizualizáció, grafikonok generálása adatpontok alapján	-	-	-
Prompt lényegi része	-		
Prezentáció készítése	-	-	-
Prompt lényegi része	-		
Egyéb (nevezze meg)			
Prompt lényegi része			
Összesített százalékos érték (a feladat érdemi részére nézve)			
Összesített érték rövid, szöveges indoklása:			